

A Simple Compiler for CCA2-Secure Pseudorandom Codes in the Standard Model

Nico Döttling¹, Antoine Joux¹, Venkata Koppula²,
Mahesh Sreekumar Rajasree², and Hendrik Waldner¹

¹CISPA Helmholtz Center for Information Security, Saarbrücken, Germany
{doettling, joux, hendrik.waldner}@cispa.de, srmahesh1994@gmail.com

²Indian Institute of Technology Delhi, New Delhi, India
kvenkata@iitd.ac.in

Abstract

Public-key pseudorandom codes (PRCs) combine two seemingly conflicting properties: their codewords are computationally indistinguishable from uniformly random strings, yet a secret-key decoder can recover the encoded message even after a bounded Hamming corruption. Security against chosen-plaintext (CPA) attacks asks that an encoding of a chosen message appears uniform. Security against adaptive chosen-ciphertext (CCA2) attacks additionally gives the adversary adaptive access to a decoding oracle for codewords outside the Hamming ball around the challenge encoding. Döttling et al. (CRYPTO 2026) showed how to realize CCA2-secure PRCs in the standard model through a generic compiler based on the Koppula–Waters hinting pseudorandom generator framework.

We give a substantially simpler black-box compiler. Starting from any adaptively α -robust, CPA-pseudorandom PRC and using only a secure PRG and an almost perfectly correct IND-CCA2-secure PKE scheme, our compiler constructs an adaptively $\alpha/2$ -robust, $\alpha/2$ -CCA2-pseudorandom PRC. The compiled PRC has twice the codeword length of the underlying PRC and therefore loses only a factor of two in its relative decoding radius; in particular, it preserves a constant relative decoding radius.

1 Introduction

Error correction and pseudorandomness impose seemingly conflicting requirements. An error-correcting code must contain enough structure to recover from noise, whereas a pseudorandom string should reveal no efficiently detectable structure. Pseudorandom codes (PRCs), introduced by Christ and Gunn [CG24], realize these seemingly contradictory properties within a single primitive: encodings appear uniform, while a secret-key decoder can recover the desired message from noisy encodings. Besides their original motivation in robust watermarking, PRCs have emerged as a useful cryptographic primitive in their own right [LSW26].

Chosen-ciphertext security. The basic public-key security experiment asks an adversary to distinguish an encoding of a chosen message from a uniformly random codeword. Chosen-ciphertext security additionally gives the adversary adaptive access to the decoder. As observed by Alrabiah

et al. [AAC⁺25a], the appropriate oracle restriction differs from that for ordinary CCA2-secure encryption. In the standard CCA2 experiment, the decryption oracle remains available after the challenge but rejects the challenge ciphertext itself. A PRC adversary, however, can change a few symbols of the challenge and obtain a different codeword that must decode to the same message. The post-challenge oracle must therefore reject an entire Hamming ball around the challenge. Outside this ball, the oracle remains available, and its answers must not reveal whether the challenge is a genuine codeword or a uniform string.

Alrabiah et al. [AAC⁺25a] introduced stronger decoding-oracle notions, including CCA2 security for public-key PRCs, and gave a construction in the random-oracle model using the Fujisaki–Okamoto transformation [FO99]. Döttling et al. [DJK⁺26a] subsequently obtained CCA2-secure PRCs in the standard model. Their generic compiler starts from an adaptively robust, CPA-secure PRC and adapts the hinting-PRG framework of Koppula and Waters [KW19]. The resulting construction uses a hinting PRG, equivocal commitments, a one-time signature scheme, pseudorandom functions, public-key encryption, and error-correcting codes. The construction presented by the authors is quite technically involved and also reduces the error-correction rate quite significantly, which leads to the following question:

Can one realize CCA2-secure PRCs in the standard model through a simple black-box compiler from a small set of conventional primitives?

1.1 Our Results

We answer this question affirmatively. Our compiler only requires a CPA-secure PRC scheme, a CCA2-secure PKE scheme and a PRG. All those primitives are used in a black-box manner and our resulting scheme loses only a factor of two in the relative decoding radius compared to the underlying PRC scheme. Informally:

Theorem 1.1 (Informal). *Let PRC_{cpa} be a public-key PRC over a finite alphabet Σ of codeword length n that is CPA-pseudorandom and adaptively α -robust. Given an almost perfectly correct IND-CCA2-secure PKE and a secure PRG, there is a black-box construction in the standard model of a public-key PRC over Σ of codeword length $2n$ that is adaptively $\frac{\alpha}{2}$ -robust and $\frac{\alpha}{2}$ -CCA2-pseudorandom.*

The three ingredients play complementary roles. The underlying PRC makes the transmitted blocks pseudorandom and robust to Hamming corruption. The IND-CCA2-secure PKE is used to achieve the CCA security of the overall construction; its ciphertexts do need not be pseudorandom or error tolerant because they will be placed inside the PRC encryption. Finally, the PRG is used to generate the underlying encoder’s randomness via a fixed-length seed. Without this compression, the length of the PKE ciphertext and the amount of randomness needed to encode that ciphertext would be defined circularly, resulting in a compiler that cannot directly be instantiated.

Our compiler is based on the CCA-secure encryption construction of Hohenberger, Lewko, and Waters [HLW12]. In a nutshell, our compiler generates two independent key pairs for the base PRC and one key pair for the PKE. To encode a message, it samples two PRG seeds, encrypts the message together with these seeds, and encodes the resulting PKE ciphertext once under each PRC key using the PRG expansion of the corresponding seed as randomness. The decoder recovers the inner ciphertext from one block, decrypts it, expands the two seeds, and re-encodes both blocks. It accepts only if the complete re-encoding is close to the received codeword. We provide a detailed technical overview in the next section.

Work	Security	Model	Robustness vs. Base ¹
Alrabiah et al. [AAC ⁺ 25a]	CCA2	ROM	α
Döttling et al. [DJK ⁺ 26a]	CCA2	Standard	$\frac{\alpha}{24}$
Chen–Mao–Yi [CMY26]	CCA1	Standard	$0.0005 \cdot \alpha$
This work	CCA2	Standard	$\frac{\alpha}{2}$

Table 1: Comparison of chosen-ciphertext-secure public-key PRC constructions. “Robustness vs. base” is the relative decoding radius retained by the compiler, expressed against the underlying PRC’s radius α .

Scope of the simplification. Compared with the compiler of Döttling et al. [DJK⁺26a], our construction requires no hinting PRG, equivocal commitment, one-time signature, PRF, or auxiliary error-correcting layer. Besides the underlying PRC, it uses only a conventional IND-CCA2-secure PKE and a standard PRG. This is a structural and proof-level simplification, not a claim that the overall assumptions are weaker or that every instantiation improves the concrete rate, key size, or running time. Our contribution is primarily conceptual: the transformation is black-box, only doubles the codeword length, and maps an underlying relative decoding radius α to $\frac{\alpha}{2}$.

1.2 Related Work

In this section, we recap the works on the general realization of PRCs, as well as works constructing CCA-secure schemes specifically.

Christ and Gunn [CG24] introduced private- and public-key PRCs and gave constructions robust to a constant rate of substitution errors. A line of subsequent work broadened both the constructions and the error models. Ghentiyala and Guruswami [GG24] gave new PRC constructions, and Golowich and Moitra [GM24] lifted PRCs robust to substitution errors into ones robust against edit errors (insertions and deletions) via an indexing technique, motivated by watermarking language models. Christ, Golowich, Gunn, Moitra, and Wicks [CGG⁺25] obtained improved pseudorandom codes from permuted puzzles. More recently, Dyseryn, Francati, and Venturi [DFV26] gave public-key PRCs from distorted McEliece assumptions, and Huang, Li, Mao, and Zhou [HLMZ26] gave high-rate public-key PRCs for edit errors.

On the negative side, Garg, Gunn, and Wang [GGW25] showed that black-box use of standard cryptographic primitives is insufficient to build PRCs, and Döttling, Müller, and Rajasree [DMR25] gave an oracle separation ruling out PRC constructions relative to local oracles.

On the security side, Alrabiah et al. [AAC⁺25a] developed the ideal-PRC viewpoint and stronger decoding-oracle notions, including public-key chosen-ciphertext security. Döttling et al. [DJK⁺26a] obtained CCA2-secure PRCs in the standard model through a generic compiler based on the Koppula–Waters framework [KW19] and separately introduced the noisy-McEliece assumption family. Concurrently, Chen, Mao, and Yi [CMY26] obtained standard-model public-key PRCs with pre-challenge CCA1 security rather than full CCA2 security. A comparison between the mentioned works that achieve CCA security as well as the compiler presented in this work is given in Table 1.

¹We highlight that this is a high-level comparison of the different results, since the robustness notion of the first two works differs from the robustness notion of the last work and is therefore not straightforwardly comparable.

Organization. We present our technical overview in Section 2. Section 3 defines the PRG, PKE, and PRC notions used in the remainder of paper. Our generic construction and its proofs of adaptive robustness and CCA2 pseudorandomness appear in Section 4.

2 Technical Overview

Our construction follows the high-level paradigm of Hohenberger, Lewko, and Waters [HLW12]. We first recall their approach and then explain how its encrypted-randomness consistency mechanism can be adapted to the PRC setting.

2.1 The Hohenberger–Lewko–Waters Paradigm

Hohenberger, Lewko, and Waters introduce *detectable chosen-ciphertext* (DCCA) security for public-key encryption. A detectable PKE scheme augments the usual algorithms with a public detection algorithm F . Given a public key, a challenge ciphertext ct^* , and another ciphertext ct , the algorithm identifies whether ct may be useful for attacking ct^* . Informally, DCCA security has two components. First, before seeing ct^* , it should be hard to produce a ciphertext that F will later classify as related to ct^* . Second, the challenge plaintext should remain hidden when the decryption oracle rejects every ciphertext that F classifies as related to ct^* .

Their generic compiler achieves full CCA2 security by combining three encryption schemes: an inner DCCA-secure scheme, an outer scheme with one-bounded chosen-ciphertext (1B-CCA) security, and an outer CPA secure scheme. Here, one-bounded security permits the adversary at most one decryption query. Setup generates independent key pairs $(\text{pk}_{\text{DCCA}}, \text{sk}_{\text{DCCA}})$, $(\text{pk}_{1\text{B-CCA}}, \text{sk}_{1\text{B-CCA}})$, and $(\text{pk}_{\text{CPA}}, \text{sk}_{\text{CPA}})$ for the three schemes.

To encrypt a message msg , the encryptor samples independent outer randomness $\mathbf{r}_{1\text{B-CCA}}$ and $\mathbf{r}_{\text{CPA}}$ and forms the inner ciphertext

$$\text{ct}_{\text{DCCA}} \leftarrow \text{Enc}_{\text{DCCA}}(\text{pk}_{\text{DCCA}}, (\text{msg}, \mathbf{r}_{1\text{B-CCA}}, \mathbf{r}_{\text{CPA}})).$$

It then encrypts this same inner ciphertext under both outer public keys using the randomness embedded in ct_{DCCA} :

$$\begin{aligned} \text{ct}_{1\text{B-CCA}} &:= \text{Enc}_{1\text{B-CCA}}(\text{pk}_{1\text{B-CCA}}, \text{ct}_{\text{DCCA}}; \mathbf{r}_{1\text{B-CCA}}), \\ \text{ct}_{\text{CPA}} &:= \text{Enc}_{\text{CPA}}(\text{pk}_{\text{CPA}}, \text{ct}_{\text{DCCA}}; \mathbf{r}_{\text{CPA}}). \end{aligned}$$

The resulting ciphertext is $(\text{ct}_{1\text{B-CCA}}, \text{ct}_{\text{CPA}})$.

To decrypt, the receiver first uses $\text{sk}_{1\text{B-CCA}}$ to recover a candidate inner ciphertext $\hat{\text{ct}}_{\text{DCCA}}$ and then decrypts it using sk_{DCCA} . If the result parses as $(\text{msg}, \mathbf{r}_{1\text{B-CCA}}, \mathbf{r}_{\text{CPA}})$, the receiver recomputes both outer ciphertexts with the recovered randomness. It outputs msg only if both recomputed ciphertexts agree with the received ones.

As observed by Hohenberger, Lewko, and Waters, this construction combines the two-key structure of Naor–Yung [NY90] with the nesting technique of Myers and Shelat [Ms09]. The encrypted randomness takes the place of a public consistency proof: after recovering the inner ciphertext, the receiver can privately verify that both outer ciphertexts were honestly generated from the same value.

Security argument. DCCA security protects the inner challenge ciphertext provided that the adversary cannot submit a useful ciphertext that the detection algorithm marks as related to it. The outer layers must therefore prevent an accepted decryption query from exposing such a related inner ciphertext. The one-bounded chosen-ciphertext-secure component and the embedded-randomness check jointly rule out this event. Moreover, a consistent ciphertext can be recovered through either outer secret key, allowing the proof to switch its decryption path.

The embedded randomness also creates the central proof difficulty: the outer challenge ciphertexts use randomness contained in the inner challenge ciphertext, so the layers cannot be changed independently in a direct hybrid argument. Hohenberger, Lewko, and Waters address this dependency through an intermediate experiment called *nested indistinguishability*. There, the inner ciphertext encrypts either the message together with the outer randomness or an all-zero string of the same length, while the outer ciphertexts are formed exactly as in the actual construction. This experiment lets the proof analyze “dangerous” decryption queries after removing the embedded information and then transfer the conclusion back to the real distribution.

2.2 Adapting the Paradigm to Pseudorandom Codes

Although our compiler shares its high-level structure with the Hohenberger–Lewko–Waters paradigm, its symmetric outer layer is, in one respect, even closer to the Naor–Yung two-key paradigm. In more detail, we rely on two independently keyed instances of the same PRC encode the same inner ciphertext and provide two decoding paths. Moreover, the randomness encrypted in the inner ciphertext can be viewed as a privately verifiable consistency proof. After recovering this randomness, or, in our final construction, the corresponding PRG seeds, the decoder re-encodes both outer blocks and accepts only if the reconstructed pair is sufficiently close to the received codeword. The recovered values thus act as a hidden consistency witness.

To realize this idea for PRCs, our compiler places a full IND-CCA2-secure PKE in the inner layer and two independently keyed instances of an adaptively robust, CPA-pseudorandom PRC over a finite alphabet Σ in the outer layer. The PKE protects the logical payload, while the PRCs turn its ciphertext into a pseudorandom codeword that remains decodable after bounded Hamming corruption.

The direct randomness-encrypting construction. The natural starting point is to generate two PRC key pairs (pk_0, sk_0) and (pk_1, sk_1) together with a PKE key pair (pk_{cca}, sk_{cca}) . To encode msg , sample independent underlying-encoder randomness $r_0, r_1 \in \{0, 1\}^{\ell_{Enc}}$ and generate

$$\begin{aligned} c_{cca} &\leftarrow \text{PKE.Enc}(pk_{cca}, (msg, r_0, r_1)), \\ c_b &:= \text{PRC}_{cpa}.\text{Encode}(pk_b, c_{cca}; r_b) \quad \text{for } b \in \{0, 1\}. \end{aligned}$$

The transmitted codeword is the pair $c := (c_0, c_1)$, represented as a codeword in Σ^{2n} .

On input $\tilde{c} = (\tilde{c}_0, \tilde{c}_1)$, the decoder first applies the underlying PRC’s decoder to $\tilde{c}_0$ and obtains a candidate inner ciphertext $\hat{c}_{cca}$. It then decrypts this candidate and parses the result as (msg, r_0, r_1) . Finally, it re-encodes $\hat{c}_{cca}$ under both public keys using the recovered randomness. Unlike the exact equality test in the PKE compiler, this robust consistency check accepts when the reconstructed pair is within the prescribed Hamming distance of $\tilde{c}$.

Security intuition. As already noted, our compiler is structurally close to the Naor–Yung paradigm [NY90]: its two outer components are independently keyed instances of the same PRC and therefore provide symmetric, interchangeable decoding paths for the same inner ciphertext. The security proof exploits precisely this two-path structure. It first filters the inner challenge ciphertext: any query recovering that ciphertext and passing the global check must lie in the Hamming ball already excluded by the CCA2 experiment. The IND-CCA2 security of the PKE can then replace the inner payload by zeros. Next, while decoder queries are answered through the first block, CPA pseudorandomness replaces the second challenge block by a uniform codeword. Adaptive robustness then permits the proof to switch to the second decoding path, after which CPA pseudorandomness replaces the first challenge block by a uniform codeword as well. Finally, the proof switches back to the original decoding path and removes the filter. Adaptive robustness ensures that the two decoding paths agree on every query accepted by the global consistency check, except with negligible probability. Thus, the second block does not improve the decoding radius; it provides the alternate path needed by this Naor–Yung-style hybrid argument.

For correctness alone, adaptive robustness of the first keyed PRC instance would suffice because the real decoder begins with the first block. The CCA2 proof, however, switches between the two decoding paths and therefore needs adaptive robustness for both independently keyed instances.

2.3 A Caveat on Instantiability

For the “direct randomness-encrypting construction” we can formally prove security and correctness as argued above. There is one small caveat that remains though.

To instantiate the PKE, we must first fix its plaintext length $\ell_{\text{msg}} + 2\ell_{\text{Enc}}$, which determines its ciphertext length ℓ_{ct} . To determine ℓ_{Enc} , however, we must first instantiate the underlying PRC for messages of exactly ℓ_{ct} bits. Thus, the PKE ciphertext length depends on the randomness length of the PRC instance, while that randomness length may itself depend on the PKE ciphertext length. This creates a circular dependency for which it is not clear how to resolve it using the mentioned primitives directly. Therefore, our above described construction is not a generally instantiable black-box compiler.

We remove this circularity using a standard PRG $G : \{0, 1\}^\lambda \rightarrow \{0, 1\}^{\ell_{\text{Enc}}}$. Instead of encrypting the full encoder randomness, the encoder samples independent seeds $\mathbf{s}_0, \mathbf{s}_1 \leftarrow \{0, 1\}^\lambda$ and computes

$$\begin{aligned} \mathbf{c}_{\text{cca}} &\leftarrow \text{PKE.Enc}(\text{pk}_{\text{cca}}, (\text{msg}, \mathbf{s}_0, \mathbf{s}_1)), \\ \mathbf{c}_b &:= \text{PRC}_{\text{cpa}}.\text{Encode}(\text{pk}_b, \mathbf{c}_{\text{cca}}; G(\mathbf{s}_b)) \quad \text{for } b \in \{0, 1\}. \end{aligned}$$

The decoder now parses the inner plaintext as $(\text{msg}, \mathbf{s}_0, \mathbf{s}_1)$ and expands the recovered seeds when reconstructing the two outer blocks.

The PKE plaintext length is now the fixed quantity $\ell_{\text{msg}} + 2\lambda$, which first determines the PKE ciphertext length ℓ_{ct} . We may then instantiate the underlying PRC for ℓ_{ct} -bit messages, determine its required randomness length ℓ_{Enc} , and choose the PRG output length accordingly. This ordering eliminates the circular dependency in the direct construction.

The proof of the final construction works as previously described with the only difference that we need to introduce an additional the strategy above by inserting a single PRG transition after the inner PKE ciphertext has been replaced. At this stage, the two challenge blocks still use $G(\mathbf{s}_0^*)$ and $G(\mathbf{s}_1^*)$ as encoder randomness. A reduction to the PRG obtains two independent samples and jointly replaces these values by independent uniform randomness $\mathbf{r}_0^*, \mathbf{r}_1^* \leftarrow \{0, 1\}^{\ell_{\text{Enc}}}$. This transition

affects only the generation of the challenge blocks: the decoder oracles continue to recover the seeds and reconstruct candidate codewords using G , exactly as in the real scheme. As already mentioned, the formal construction and proof are presented in Section 4.

3 Preliminaries

Let PPT denote probabilistic polynomial-time. A function $\nu : \mathbb{N} \rightarrow [0, 1]$ is *negligible* if, for every positive polynomial $p(\cdot)$, there exists $\lambda_0 \in \mathbb{N}$ such that $\nu(\lambda) < 1/p(\lambda)$ for every $\lambda \geq \lambda_0$. We write $\text{negl}(\lambda)$ for an unspecified negligible function. Let $[n] := \{1, \dots, n\}$. We write binary strings and codewords over Σ in bold font and structured values as tuples. Whenever an algorithm expects a bit string, a tuple denotes its canonical fixed-length binary encoding. For a finite alphabet Σ and $\mathbf{x}, \mathbf{y} \in \Sigma^N$, we denote their Hamming distance by $\text{Ham}(\mathbf{x}, \mathbf{y}) := |\{i \in [N] : x_i \neq y_i\}|$. For a finite set S , the notation $x \leftarrow S$ means that x is sampled uniformly from S .

3.1 Pseudorandom Generators

Pseudorandom generators are used to expand short uniform seeds into longer strings that are computationally indistinguishable from random. We recall their standard syntax and security notion.

Definition 3.1 (Pseudorandom Generator [BM84, Yao82]). *Let $\ell_{\text{Enc}} = \ell_{\text{Enc}}(\lambda) > \lambda$ be polynomially bounded. A pseudorandom generator (PRG) is a deterministic PPT algorithm G with the following syntax.*

$G(\mathbf{s})$: *The generator takes as input a seed $\mathbf{s} \in \{0, 1\}^\lambda$ and outputs a string $\mathbf{r} \in \{0, 1\}^{\ell_{\text{Enc}}}$.*

For $b \in \{0, 1\}$, we define the following sampling oracle $\mathcal{O}_b^{\text{prg}}$

$$\mathcal{O}_b^{\text{prg}}() := \begin{cases} G(\mathbf{s}) & \text{with } \mathbf{s} \leftarrow \{0, 1\}^\lambda & \text{if } b = 0, \\ \mathbf{r} & \text{with } \mathbf{r} \leftarrow \{0, 1\}^{\ell_{\text{Enc}}} & \text{if } b = 1. \end{cases}$$

For a PPT adversary $\mathcal{A}$ making polynomially many oracle queries, we define its advantage as

$$\text{Adv}_{G, \mathcal{A}}^{\text{prg}}(\lambda) := \left| \Pr[\mathcal{A}^{\mathcal{O}_0^{\text{prg}}}(1^\lambda) = 1] - \Pr[\mathcal{A}^{\mathcal{O}_1^{\text{prg}}}(1^\lambda) = 1] \right|,$$

where the probabilities are taken over the randomness of $\mathcal{A}$ and the fresh randomness used by each oracle invocation. We say that G is secure if this advantage is negligible for every PPT adversary $\mathcal{A}$ making polynomially many oracle queries.

3.2 Public Key encryption

In this section, we recall the syntax and correctness requirements of public-key encryption, followed by its standard CCA2-security notion.

Definition 3.2 (Public-Key Encryption). *A public-key encryption scheme (PKE) is a triple of PPT algorithms $\text{PKE} = (\text{Setup}, \text{Enc}, \text{Dec})$:*

$\text{Setup}(1^\lambda, n)$: *The setup algorithm takes as input a unary representation of the security parameter λ and the message length n and outputs a public key pk and secret key sk .*

$\text{Enc}(\text{pk}, m)$: The encryption algorithm takes as input a public key pk and a message $m \in \{0, 1\}^n$ and outputs a ciphertext ct .

$\text{Dec}(\text{sk}, \text{ct})$: The decryption algorithm takes as input a secret key sk and a ciphertext ct and outputs either a message $m \in \{0, 1\}^n$ or $\perp$.

We say that a scheme is almost perfectly correct if there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda, n \in \mathbb{N}$, for a $1 - \text{negl}(\lambda)$ fraction of keys pk, sk in the support of Setup , for all $m \in \{0, 1\}^n$, it holds that

$$\Pr[\text{Dec}(\text{sk}, \text{Enc}(\text{pk}, m)) = m] = 1,$$

where the probability is taken over the randomness of Enc .

Remark 3.3 (Conventions for encryption and encoding). Throughout, we assume that every PKE scheme we use is length regular, that is, its ciphertext length depends only on the security parameter and the message length, and has deterministic decryption, with Dec returning $\perp$ on any input outside the ciphertext space. For a randomized algorithm (an encryptor or a PRC encoder), we write $\text{Enc}(\text{pk}, \mathbf{x}; \mathbf{r})$ and $\text{Encode}(\text{pk}, \mathbf{x}; \mathbf{r})$ for the deterministic evaluation of the algorithm on input $\mathbf{x}$ with explicit randomness $\mathbf{r}$.

Definition 3.4 (CCA2 Security [RS92]). For a PPT adversary $\mathcal{A}$ and bit $b \in \{0, 1\}$, consider the following experiment.

$\text{Exp}_{\text{PRC}, \mathcal{A}}^{\text{cca2}, b}(\lambda)$:

$(\text{pk}, \text{sk}) \leftarrow \text{Setup}(1^\lambda, 1^n)$

$(m_0, m_1) \leftarrow \mathcal{A}^{\text{Dec}(\text{sk}, \cdot)}(\text{pk})$, where $m_0, m_1 \in \{0, 1\}^n$

$\text{ct} \leftarrow \text{Enc}(\text{pk}, m_b)$

$b' \leftarrow \mathcal{A}^{\mathcal{O}_{\text{sk}, \text{ct}}(\cdot)}(\text{ct})$ where

$$\mathcal{O}_{\text{sk}, \text{ct}}(\text{ct}') = \begin{cases} \text{Dec}(\text{sk}, \text{ct}') & \text{if } \text{ct}' \neq \text{ct} \\ \perp & \text{otherwise} \end{cases}.$$

return b' .

We define the advantage of $\mathcal{A}$ as

$$\text{Adv}_{\text{PKE}, \mathcal{A}}^{\text{cca2}}(\lambda) := \left| \Pr[\text{Exp}_{\text{PKE}, \mathcal{A}}^{\text{cca2}, 0}(\lambda) = 1] - \Pr[\text{Exp}_{\text{PKE}, \mathcal{A}}^{\text{cca2}, 1}(\lambda) = 1] \right|.$$

We say that PKE is CCA2-secure if this advantage is negligible for every PPT adversary $\mathcal{A}$.

In addition to CCA security, we require our PKE scheme to satisfy a property that we call *unpredictability*. Informally, unpredictability states that, for any efficiently computable plaintext family $\mathbf{x} = \mathbf{x}(\lambda)$, even given the public key and the plaintext, the probability of predicting the exact ciphertext produced using fresh encryption randomness is negligible. This probability is taken over an average generated key-pair.

This type of requirement was originally introduced in the work of Fujisaki and Okamoto [FO13], who defined spreadness. Their notion requires that, for *every* public key in the support of key generation and every plaintext, the probability of any particular ciphertext occurring under fresh encryption randomness is negligible. Consequently, spreadness implies our unpredictability notion. It is, however, a stronger requirement, since it imposes a worst-case bound over both public keys and plaintexts.

Don et al. [DFMS22] subsequently introduced weak spreadness, which relaxes the worst-case requirement over public keys to an average over key generation. Weak spreadness is therefore closer to our unpredictability notion, but the two notions are not identical: weak spreadness still maximizes over all plaintexts for each sampled public key, whereas our notion considers each fixed, efficiently computable plaintext family. Therefore, weak spreadness also implies unpredictability.

In Section A, we show that unpredictability is satisfied by standard IND-CPA-secure PKE schemes. Formally:

Lemma 3.5 (Fresh-ciphertext unpredictability). *Let PKE be an almost perfectly correct, IND-CPA-secure PKE with deterministic decryption whose plaintext space contains at least two messages. For an efficiently computable plaintext family $\mathbf{x} = \mathbf{x}(\lambda)$, define*

$$p_{\max}(\mathbf{pk}, \mathbf{x}) := \max_{\mathbf{z} \in \{0,1\}^{\ell_{\text{ct}}}} \Pr_{\mathbf{r} \leftarrow \{0,1\}^{\ell_{\text{rand}}}} [\text{PKE.Enc}(\mathbf{pk}, \mathbf{x}; \mathbf{r}) = \mathbf{z}].$$

Then $\mathbb{E}_{(\mathbf{pk}, \mathbf{sk}) \leftarrow \text{PKE.Setup}(1^\lambda, |\mathbf{x}|)} [p_{\max}(\mathbf{pk}, \mathbf{x})] = \text{negl}(\lambda)$.

3.3 Pseudorandom Codes

Pseudorandom codes were introduced by Christ and Gunn [CG24]. We recall their syntax, adaptive robustness, and their CPA as well as CCA2 security. We follow the syntax of Döttling et al. [DJK⁺26a].

Definition 3.6 (Pseudorandom codes). *A public-key pseudorandom code over finite alphabet $\Sigma = \Sigma(\lambda)$ consists of three PPT algorithms $\text{PRC} = (\text{Setup}, \text{Encode}, \text{Decode})$ with message length $n = n(\lambda)$ and codeword length $N = N(\lambda)$. The algorithms have the following syntax.*

$\text{Setup}(1^\lambda, n)$: *The setup algorithm takes as input a unary representation of the security parameter λ and message length n and outputs a key pair $(\mathbf{pk}, \mathbf{sk})$.*

$\text{Encode}(\mathbf{pk}, \text{msg})$: *the encoding algorithm takes as input a public key $\mathbf{pk}$ and a message $m \in \{0,1\}^n$ and outputs a codeword $\mathbf{c} \in \Sigma^N$.*

$\text{Decode}(\mathbf{sk}, \tilde{\mathbf{c}})$: *The decoding algorithm is deterministic that takes as input a secret key $\mathbf{sk}$ and a string $\tilde{\mathbf{c}} \in \Sigma^N$ and outputs either a message $\text{msg} \in \mathcal{M}$ or $\perp$.*

We require robustness: for every $\text{msg} \in \{0,1\}^n$,

$$\Pr[\text{Decode}(\mathbf{sk}, \text{Encode}(\mathbf{pk}, \text{msg})) \neq \text{msg}] \leq \text{negl}(\lambda),$$

where the probability is over the randomness of Setup and Encode.

Definition 3.7 (Adaptive Robustness [AAC⁺25b]). *Let PRC be a public-key PRC over a finite alphabet Σ with message space $\{0,1\}^n$, codeword length N , and randomness space $\mathcal{R} = \mathcal{R}(\lambda)$ for its encoder. For $\tau = \tau(\lambda) \in [0,1]$ and a PPT adversary $\mathcal{A}$, consider the following experiment.*

1. *The challenger samples $(\mathbf{pk}, \mathbf{sk}) \leftarrow \text{Setup}(1^\lambda)$ and gives $\mathbf{pk}$ to $\mathcal{A}$.*
2. *The adversary outputs $(\text{msg}, \mathbf{r}, \tilde{\mathbf{c}}) \leftarrow \mathcal{A}(\mathbf{pk})$, where $\text{msg} \in \mathcal{M}$, $\mathbf{r} \in \mathcal{R}$, and $\tilde{\mathbf{c}} \in \Sigma^N$.*
3. *The challenger computes $\mathbf{c} := \text{Encode}(\mathbf{pk}, \text{msg}; \mathbf{r})$ and $\text{msg}' := \text{Decode}(\mathbf{sk}, \tilde{\mathbf{c}})$. The adversary wins if $\text{Ham}(\mathbf{c}, \tilde{\mathbf{c}}) \leq \tau N$ and $\text{msg}' \neq \text{msg}$.*

We say that PRC is adaptively τ -robust if every PPT adversary wins this experiment with negligible probability.

Definition 3.8 (CPA Pseudorandomness [CG24]). Let $\text{PRC} = (\text{Setup}, \text{Encode}, \text{Decode})$ be a public-key PRC over a finite alphabet Σ , with message space $\{0, 1\}^n$ and codeword length N . For $b \in \{0, 1\}$, $n = \text{poly}(\lambda)$ and a two-stage PPT adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$, define the following experiment.

```

ExpPRC, Acpa, b( $\lambda$ ):
  (pk, sk)  $\leftarrow$  Setup( $1^\lambda, n$ )
  (msg*, state)  $\leftarrow$   $\mathcal{A}_1$ (pk), where msg*  $\in \{0, 1\}^n$ 
  if  $b = 0$ :  $\mathbf{c}^* \leftarrow \text{Encode}(\text{pk}, \text{msg}^*)$ ;
  if  $b = 1$ :  $\mathbf{c}^* \leftarrow \Sigma^N$ 
   $b' \leftarrow \mathcal{A}_2(\text{state}, \mathbf{c}^*)$ 
  return  $b'$ .

```

We define the advantage of $\mathcal{A}$ as

$$\text{Adv}_{\text{PRC}, \mathcal{A}}^{\text{cpa}}(\lambda) := \left| \Pr[\text{Exp}_{\text{PRC}, \mathcal{A}}^{\text{cpa}, 0}(\lambda) = 1] - \Pr[\text{Exp}_{\text{PRC}, \mathcal{A}}^{\text{cpa}, 1}(\lambda) = 1] \right|.$$

We say that PRC is CPA-pseudorandom if this advantage is negligible for every two-stage PPT adversary $\mathcal{A}$.

Definition 3.9 (CCA2 Pseudorandomness [AAC⁺25b, DJK⁺26b]). Let PRC be a PRC over a finite alphabet Σ with message space $\mathcal{M}$, codeword length N , and let $\beta = \beta(\lambda) \in [0, 1]$ be the challenge-exclusion rate. For $b \in \{0, 1\}$ and an adversary $\mathcal{A}$, define the following experiment.

```

ExpPRC, A, \betacca2, b( $\lambda$ ):
  (pk, sk)  $\leftarrow$  Setup( $1^\lambda$ )
  (msg*, state)  $\leftarrow \mathcal{A}^{\text{Decode}(\text{sk}, \cdot)}(\text{pk})$ , where msg*  $\in \mathcal{M}$ 
  if  $b = 0$ :  $\mathbf{c}^* \leftarrow \text{Encode}(\text{pk}, \text{msg}^*)$ ; if  $b = 1$ :  $\mathbf{c}^* \leftarrow \Sigma^N$ 
   $b' \leftarrow \mathcal{A}^{\mathcal{O}_{\mathbf{c}^*, \text{sk}, \beta}(\cdot)}(\text{state}, \mathbf{c}^*)$  where
    
$$\mathcal{O}_{\mathbf{c}^*, \text{sk}, \beta}(\tilde{\mathbf{c}}) = \begin{cases} \text{Decode}(\text{sk}, \tilde{\mathbf{c}}) & \text{if } \text{Ham}(\tilde{\mathbf{c}}, \mathbf{c}^*) > \beta N, \\ \perp & \text{otherwise.} \end{cases}$$

  return  $b'$ .

```

We define the advantage of $\mathcal{A}$ as

$$\text{Adv}_{\text{PRC}, \mathcal{A}, \beta}^{\text{cca2}}(\lambda) := \left| \Pr[\text{Exp}_{\text{PRC}, \mathcal{A}, \beta}^{\text{cca2}, 0}(\lambda) = 1] - \Pr[\text{Exp}_{\text{PRC}, \mathcal{A}, \beta}^{\text{cca2}, 1}(\lambda) = 1] \right|.$$

We say that PRC is β -CCA2-pseudorandom if this advantage is negligible for every PPT adversary making polynomially many oracle queries.

4 A Generic CCA2 Construction from Minimal Assumptions

In this section, we provide our construction of a CCA2-secure PRC. As already mentioned, our construction requires an adaptively robust, CPA-secure PRC PRC_{cpa} , an almost perfectly correct IND-CCA2-secure public-key encryption scheme PKE, and a standard PRG G . We follow the conventions of Remark 3.3: PKE is length regular, has deterministic decryption, and PKE.Dec returns $\perp$ on inputs outside its ciphertext space. We write $\text{PKE.Enc}(\text{pk}, \mathbf{x}; \mathbf{r})$ and $\text{PRC}_{\text{cpa}}.\text{Encode}(\text{pk}, \mathbf{x}; \mathbf{r})$ for evaluation on explicit randomness.

Parameters and primitives. Let $n = n(\lambda)$, $\ell_{\text{ct}} = \ell_{\text{ct}}(\lambda)$, $\ell_{\text{Enc}} = \ell_{\text{Enc}}(\lambda)$, $\ell_{\text{rand}} = \ell_{\text{rand}}(\lambda)$, and $\ell_{\text{msg}} = \ell_{\text{msg}}(\lambda)$ be positive, polynomially bounded integers. Let $\mathcal{X} := \{0, 1\}^{\ell_{\text{msg}}} \times \{0, 1\}^\lambda \times \{0, 1\}^\lambda$ denote the structured PKE plaintext space and let $\mathbf{x}_0 := (0^{\ell_{\text{msg}}}, 0^\lambda, 0^\lambda) \in \mathcal{X}$ denote its all-zero element. As specified in Remark 3.3, these tuples use their canonical fixed-length binary encoding when passed to the PKE.

- $\text{PRC}_{\text{cpa}} = (\text{PRC}_{\text{cpa}}.\text{Setup}, \text{PRC}_{\text{cpa}}.\text{Encode}, \text{PRC}_{\text{cpa}}.\text{Decode})$ is a PRC over a finite alphabet Σ with message space $\{0, 1\}^{\ell_{\text{ct}}}$ and codeword length n , whose randomness is of size ℓ_{Enc} . We assume that PRC_{cpa} is CPA-pseudorandom and adaptively α -robust for some $\alpha = \alpha(\lambda) \in [0, 1]$.
- $G : \{0, 1\}^\lambda \rightarrow \{0, 1\}^{\ell_{\text{Enc}}}$ is a secure PRG.
- $\text{PKE} = (\text{PKE}.\text{Setup}, \text{PKE}.\text{Enc}, \text{PKE}.\text{Dec})$ is an almost perfectly correct, IND-CCA2-secure PKE with structured plaintext space $\mathcal{X}$, ciphertext space $\{0, 1\}^{\ell_{\text{ct}}}$, and ℓ_{rand} randomness.

The constructed PRC is over Σ , has message space $\{0, 1\}^{\ell_{\text{msg}}}$, and has codeword length $N := 2n$. Its decoder uses the reconstruction radius $\lfloor \alpha n \rfloor$, corresponding to relative radius $\lfloor \alpha n \rfloor / N = \lfloor \alpha n \rfloor / (2n) \leq \alpha/2$.

Construction. The PRC $\text{PRC}_{\text{cca}} = (\text{Setup}, \text{Encode}, \text{Decode})$ is realized as follows.

Setup($1^\lambda, \ell_{\text{msg}}$): The setup algorithm performs the following steps.

1. For each $b \in \{0, 1\}$, sample an underlying-PRC key pair $(\text{pk}_b, \text{sk}_b) \leftarrow \text{PRC}_{\text{cpa}}.\text{Setup}(1^\lambda, \ell_{\text{ct}})$.
2. Sample a PKE key pair $(\text{pk}_{\text{cca}}, \text{sk}_{\text{cca}}) \leftarrow \text{PKE}.\text{Setup}(1^\lambda, \ell_{\text{msg}} + 2\lambda)$.
3. Output the public key $\text{pk} = (\text{pk}_0, \text{pk}_1, \text{pk}_{\text{cca}})$ and the secret key $\text{sk} = (\text{sk}_0, \text{sk}_{\text{cca}}, \text{pk}_0, \text{pk}_1)$.

Encode(pk, msg): The encoding algorithm parses $\text{pk} = (\text{pk}_0, \text{pk}_1, \text{pk}_{\text{cca}})$ and performs the following steps.

1. Sample $\mathbf{r}_{\text{pke}} \leftarrow \{0, 1\}^{\ell_{\text{rand}}}$ and $\mathbf{s}_0, \mathbf{s}_1 \leftarrow \{0, 1\}^\lambda$ independently and uniformly, and set $\mathbf{r} := (\mathbf{r}_{\text{pke}}, \mathbf{s}_0, \mathbf{s}_1)$.
2. Compute $\mathbf{c}_{\text{cca}} \leftarrow \text{PKE}.\text{Enc}(\text{pk}_{\text{cca}}, (\text{msg}, \mathbf{s}_0, \mathbf{s}_1); \mathbf{r}_{\text{pke}})$.
3. For every $b \in \{0, 1\}$, compute $\mathbf{c}_b \leftarrow \text{PRC}_{\text{cpa}}.\text{Encode}(\text{pk}_b, \mathbf{c}_{\text{cca}}; G(\mathbf{s}_b))$.
4. Output the block pair $\mathbf{c} := (\mathbf{c}_0, \mathbf{c}_1)$, canonically represented as an element of Σ^N .

Decode($\text{sk}, \tilde{\mathbf{c}}$): The decoding algorithm parses $\text{sk} = (\text{sk}_0, \text{sk}_{\text{cca}}, \text{pk}_0, \text{pk}_1)$ and $\tilde{\mathbf{c}} = (\tilde{\mathbf{c}}_0, \tilde{\mathbf{c}}_1)$ with $\tilde{\mathbf{c}}_0, \tilde{\mathbf{c}}_1 \in \Sigma^n$, and performs the following steps.

1. Compute $\hat{\mathbf{c}}_{\text{cca}} \leftarrow \text{PRC}_{\text{cpa}}.\text{Decode}(\text{sk}_0, \tilde{\mathbf{c}}_0)$. If $\hat{\mathbf{c}}_{\text{cca}} = \perp$, output $\perp$.
2. Compute $\mathbf{x} \leftarrow \text{PKE}.\text{Dec}(\text{sk}_{\text{cca}}, \hat{\mathbf{c}}_{\text{cca}})$. If $\mathbf{x} = \perp$ or $\mathbf{x}$ cannot be parsed as a tuple $(\text{msg}, \mathbf{s}_0, \mathbf{s}_1) \in \mathcal{X}$, output $\perp$.
3. For every $b \in \{0, 1\}$, compute $\hat{\mathbf{c}}_b \leftarrow \text{PRC}_{\text{cpa}}.\text{Encode}(\text{pk}_b, \hat{\mathbf{c}}_{\text{cca}}; G(\mathbf{s}_b))$ and set $\hat{\mathbf{c}} := (\hat{\mathbf{c}}_0, \hat{\mathbf{c}}_1)$. If $\text{Ham}(\hat{\mathbf{c}}, \tilde{\mathbf{c}}) \leq \lfloor \alpha n \rfloor$, output msg . Otherwise output $\perp$.

Lemma 4.1 (Adaptive robustness). *If PRC_{cpa} is adaptively α -robust and PKE is almost perfectly correct, then PRC_{cca} is adaptively $\frac{\alpha}{2}$ -robust.*

Proof. Suppose toward a contradiction that a PPT adversary $\mathcal{A}$ wins the adaptive $\frac{\alpha}{2}$ -robustness game against PRC_{cca} with non-negligible probability. We construct a PPT adversary $\mathcal{B}$ against the adaptive α -robustness of PRC_{cpa} .

Given a public key pk_0 from its challenger, $\mathcal{B}$ samples $(\text{pk}_1, \text{sk}_1) \leftarrow \text{PRC}_{\text{cpa}}.\text{Setup}(1^\lambda, \ell_{\text{ct}})$ and $(\text{pk}_{\text{cca}}, \text{sk}_{\text{cca}}) \leftarrow \text{PKE}.\text{Setup}(1^\lambda, \ell_{\text{msg}} + 2\lambda)$, discards sk_1 , and gives $\text{pk} = (\text{pk}_0, \text{pk}_1, \text{pk}_{\text{cca}})$ to $\mathcal{A}$. Let $(\text{msg}, \mathbf{r}, \tilde{\mathbf{c}}) \leftarrow \mathcal{A}(\text{pk})$ be $\mathcal{A}$'s output, parsed as $\mathbf{r} = (\mathbf{r}_{\text{pke}}, \mathbf{s}_0, \mathbf{s}_1)$ and $\tilde{\mathbf{c}} = (\tilde{\mathbf{c}}_0, \tilde{\mathbf{c}}_1)$. Then $\mathcal{B}$ sets $\mathbf{c}_{\text{cca}} := \text{PKE}.\text{Enc}(\text{pk}_{\text{cca}}, (\text{msg}, \mathbf{s}_0, \mathbf{s}_1); \mathbf{r}_{\text{pke}})$ and outputs $(\mathbf{c}_{\text{cca}}, G(\mathbf{s}_0), \tilde{\mathbf{c}}_0)$ in its own game.

Assume the honestly generated PKE key pair is one on which decryption is perfectly correct. By almost perfect correctness this fails only with negligible probability. Suppose that $\mathcal{A}$ wins, i.e., for $\mathbf{c} := (\mathbf{c}_0, \mathbf{c}_1)$ with $\mathbf{c}_b := \text{PRC}_{\text{cpa}}.\text{Encode}(\text{pk}_b, \mathbf{c}_{\text{cca}}; G(\mathbf{s}_b))$ we have $\text{Ham}(\mathbf{c}, \tilde{\mathbf{c}}) \leq \lfloor \alpha n \rfloor$ and $\text{Decode}(\text{sk}, \tilde{\mathbf{c}}) \neq \text{msg}$. In particular, $N = 2n$ implies that the permitted relative error rate $\alpha/2$ for the compiled scheme corresponds to at most $\lfloor (\alpha/2)N \rfloor = \lfloor \alpha n \rfloor$ errors. Projecting onto the first length- n block cannot increase Hamming distance, and therefore $\text{Ham}(\mathbf{c}_0, \tilde{\mathbf{c}}_0) \leq \text{Ham}(\mathbf{c}, \tilde{\mathbf{c}}) \leq \lfloor \alpha n \rfloor$, so $\mathcal{B}$'s corrupted codeword $\tilde{\mathbf{c}}_0$ is within the underlying scheme's relative radius α , namely αn , of $\text{PRC}_{\text{cpa}}.\text{Encode}(\text{pk}_0, \mathbf{c}_{\text{cca}}; G(\mathbf{s}_0))$. The first block is the critical one because the compiled decoder applies the underlying decoder only to $\tilde{\mathbf{c}}_0$; the second block is used only in the subsequent global consistency check.

It remains to show that the underlying decoder does not recover the encoded message $\mathbf{c}_{\text{cca}}$ from $\tilde{\mathbf{c}}_0$. Together with the distance bound above, this establishes that $\mathcal{B}$ wins its adaptive robustness game. Suppose, towards a contradiction, that $\text{PRC}_{\text{cpa}}.\text{Decode}(\text{sk}_0, \tilde{\mathbf{c}}_0) = \mathbf{c}_{\text{cca}}$. Then perfect correctness of PKE would let the compiled decoder recover $(\text{msg}, \mathbf{s}_0, \mathbf{s}_1)$, reconstruct $\hat{\mathbf{c}}_b = \mathbf{c}_b$ using $G(\mathbf{s}_b)$ for both $b \in \{0, 1\}$, and hence $\hat{\mathbf{c}} = \mathbf{c}$. The global check would pass since $\text{Ham}(\hat{\mathbf{c}}, \tilde{\mathbf{c}}) = \text{Ham}(\mathbf{c}, \tilde{\mathbf{c}}) \leq \lfloor \alpha n \rfloor$, forcing $\text{Decode}(\text{sk}, \tilde{\mathbf{c}}) = \text{msg}$, a contradiction. Thus $\mathcal{B}$ wins whenever $\mathcal{A}$ wins and the PKE key is good, contradicting the adaptive robustness of PRC_{cpa} . $\square$ $\square$

CCA2 pseudorandomness. In this section, we state the main theorem, describe the hybrid experiments, and then establish each transition in a separate lemma.

Theorem 4.2 (CCA2 pseudorandomness). *Let PRC_{cpa} be an adaptively α -robust, CPA-pseudorandom PRC over a finite alphabet Σ . Let G be a secure PRG, and let PKE be an almost perfectly correct IND-CCA2-secure PKE. Then the constructed scheme PRC_{cca} is an $\frac{\alpha}{2}$ -CCA2-pseudorandom PRC over Σ .*

Proof. Fix a PPT adversary $\mathcal{A}$ making at most $q = q(\lambda)$ decoder queries, and set $t := \lfloor \alpha n \rfloor$ and $\beta := \alpha/2$. Since $N = 2n$, we have $t \leq \beta N$, so every query accepted by the constructed decoder around some point is within the challenge-exclusion ball of that point.

Auxiliary decoder. The hybrid challenger runs **Setup** honestly but additionally retains sk_1 as private state. This does not change the secret key $\text{sk} = (\text{sk}_0, \text{sk}_{\text{cca}}, \text{pk}_0, \text{pk}_1)$ handed to the experiment. For each $b \in \{0, 1\}$, define the branch-specific decoder key

$$\text{sk}^{(b)} := (\text{sk}_b, \text{sk}_{\text{cca}}, \text{pk}_0, \text{pk}_1).$$

In particular, $\text{sk}^{(0)} = \text{sk}$. For $b \in \{0, 1\}$ and a filter set $F \subseteq \{0, 1\}^{\ell_{\text{ct}}}$, we use the auxiliary decoder $\text{Decode}_b^F(\text{sk}^{(b)}, \cdot)$ specified in Fig. 1.

$$\text{Decode}_b^F(\text{sk}^{(b)}, \tilde{\mathbf{c}})$$

1. Compute $\mathbf{z} \leftarrow \text{PRC}_{\text{cpa}}.\text{Decode}(\text{sk}_b, \tilde{\mathbf{c}}_b)$. If $\mathbf{z} = \perp$ or $\mathbf{z} \in F$, output $\perp$.
2. Compute $\mathbf{x} \leftarrow \text{PKE}.\text{Dec}(\text{sk}_{\text{cca}}, \mathbf{z})$. If $\mathbf{x} = \perp$ or $\mathbf{x}$ cannot be parsed as $(\text{msg}, \mathbf{s}_0, \mathbf{s}_1) \in \mathcal{X}$, output $\perp$.
3. For each $j \in \{0, 1\}$, compute $\hat{\mathbf{c}}_j \leftarrow \text{PRC}_{\text{cpa}}.\text{Encode}(\text{pk}_j, \mathbf{z}; G(\mathbf{s}_j))$, and set $\hat{\mathbf{c}} := (\hat{\mathbf{c}}_0, \hat{\mathbf{c}}_1)$.
4. If $\text{Ham}(\hat{\mathbf{c}}, \tilde{\mathbf{c}}) \leq t$, output msg . Otherwise, output $\perp$.

Figure 1: The auxiliary decoder used in the hybrid proof.

Thus, for every $\tilde{\mathbf{c}} \in \Sigma^N$, $\text{Decode}_0^\emptyset(\tilde{\mathbf{c}}) = \text{Decode}(\text{sk}, \tilde{\mathbf{c}})$. In every hybrid the post-challenge oracle first applies the outer exclusion test of Definition 3.9 (returning $\perp$ whenever $\text{Ham}(\tilde{\mathbf{c}}, \mathbf{c}^*) \leq \beta N$) and only then invokes the stated decoder.

For notational simplicity, in the remainder of the proof we omit arguments that are clear from context. In particular, Decode_b^F denotes the auxiliary decoder itself, while $\text{Decode}_b^F(\tilde{\mathbf{c}})$ abbreviates $\text{Decode}_b^F(\text{sk}^{(b)}, \tilde{\mathbf{c}})$ whenever the branch-specific decoder key is clear.

Now, we define the hybrids.

Hybrid $\mathcal{H}_0$: This is the real experiment $\text{Exp}_{\text{PRC}_{\text{cca}}, \mathcal{A}, \beta}^{\text{cca2}, 0}$, written out in full. The decoder oracle uses $\text{Decode}_0^\emptyset$, which is the real decoder $\text{Decode}(\text{sk}, \cdot)$, in both phases. The experiment proceeds as follows.

Initialization phase: The challenger performs the following steps.

1. For each $b \in \{0, 1\}$, sample $(\text{pk}_b, \text{sk}_b) \leftarrow \text{PRC}_{\text{cpa}}.\text{Setup}(1^\lambda, \ell_{\text{ct}})$.
2. Sample $(\text{pk}_{\text{cca}}, \text{sk}_{\text{cca}}) \leftarrow \text{PKE}.\text{Setup}(1^\lambda, \ell_{\text{msg}} + 2\lambda)$.
3. Set $\text{pk} = (\text{pk}_0, \text{pk}_1, \text{pk}_{\text{cca}})$ and $\text{sk} = (\text{sk}_0, \text{sk}_{\text{cca}}, \text{pk}_0, \text{pk}_1)$, additionally retaining sk_1 as private state.
4. Send pk to $\mathcal{A}$.

Pre-challenge phase: The adversary $\mathcal{A}$ adaptively makes polynomially many decoder queries. Each query $\tilde{\mathbf{c}} = (\tilde{\mathbf{c}}_0, \tilde{\mathbf{c}}_1)$ is answered by $\text{Decode}_0^\emptyset(\tilde{\mathbf{c}})$, that is, the challenger runs the real decoder in full: it decodes $\tilde{\mathbf{c}}_0$ under sk_0 using the underlying PRC, decrypts under sk_{cca} , reconstructs $\hat{\mathbf{c}}$, and returns msg iff $\text{Ham}(\hat{\mathbf{c}}, \tilde{\mathbf{c}}) \leq t$. No challenge-exclusion test is applied in this phase.

Challenge phase: $\mathcal{A}$ outputs a message $\text{msg}^* \in \{0, 1\}^{\ell_{\text{msg}}}$. The challenger samples $\mathbf{r}_{\text{pke}}^* \leftarrow \{0, 1\}^{\ell_{\text{rand}}}$ and $\mathbf{s}_0^*, \mathbf{s}_1^* \leftarrow \{0, 1\}^\lambda$, and sets

$$\begin{aligned} \mathbf{x}^* &:= (\text{msg}^*, \mathbf{s}_0^*, \mathbf{s}_1^*), & \mathbf{c}_{\text{cca}}^* &:= \text{PKE}.\text{Enc}(\text{pk}_{\text{cca}}, \mathbf{x}^*; \mathbf{r}_{\text{pke}}^*), \\ \mathbf{c}_b^* &:= \text{PRC}_{\text{cpa}}.\text{Encode}(\text{pk}_b, \mathbf{c}_{\text{cca}}^*; G(\mathbf{s}_b^*)) & \text{for } b \in \{0, 1\}, \end{aligned}$$

and returns the challenge codeword $\mathbf{c}^* := (\mathbf{c}_0^*, \mathbf{c}_1^*)$ to $\mathcal{A}$.

Post-challenge phase: $\mathcal{A}$ adaptively makes polynomially many further decoder queries. On a query $\tilde{\mathbf{c}}$, the challenger *first* applies the outer exclusion test: if $\text{Ham}(\tilde{\mathbf{c}}, \mathbf{c}^*) \leq \beta N$ it returns $\perp$. *Otherwise* it returns $\text{Decode}_0^\emptyset(\tilde{\mathbf{c}})$. Recall $t \leq \beta N$, so the internal radius- t acceptance of the decoder is strictly inside the excluded ball.

Response phase: $\mathcal{A}$ outputs a bit b' , which is the output of the experiment.

The total number of decoder queries across both phases is at most q . Since $\text{Decode}_0^\emptyset = \text{Decode}(\text{sk}, \cdot)$ and retaining sk_1 as private state leaves the secret key sk handed to the experiment unchanged, this is exactly $\text{Exp}_{\text{PRC}_{\text{cca}}, \mathcal{A}, \beta}^{\text{cca2}, 0}$.

Hybrid $\mathcal{H}_1$: This hybrid proceeds as $\mathcal{H}_0$ except that the post-challenge oracle uses $\text{Decode}_0^{\{\mathbf{c}_{\text{cca}}^*\}}$ instead of $\text{Decode}_0^\emptyset$. The indistinguishability of this and the previous hybrid follows from almost perfect correctness of PKE, and is proven in Lemma 4.3.

Hybrid $\mathcal{H}_2$: This hybrid proceeds as $\mathcal{H}_1$ except that the challenge PKE plaintext $\mathbf{x}^*$ is replaced by the zero tuple $\mathbf{x}_0$. The independently sampled seeds $\mathbf{s}_0^*, \mathbf{s}_1^*$ are still expanded to form the two outer challenge blocks, and the filter $\{\mathbf{c}_{\text{cca}}^*\}$ remains in place. The indistinguishability of this and the previous hybrid follows from the IND-CCA2 security of PKE, and is proven in Lemma 4.4.

Hybrid $\mathcal{H}_3$: This hybrid proceeds as $\mathcal{H}_2$ except that, for both $b \in \{0, 1\}$, the randomness $G(\mathbf{s}_b^*)$ used to form the b -th challenge block is replaced by independent uniform randomness $\mathbf{r}_b^* \leftarrow \{0, 1\}^{\ell_{\text{Enc}}}$. Thus $\mathbf{c}_b^* := \text{PRC}_{\text{cpa}}.\text{Encode}(\text{pk}_b, \mathbf{c}_{\text{cca}}^*; \mathbf{r}_b^*)$ for both $b \in \{0, 1\}$. The change affects only the generation of the challenge blocks; every decoder oracle continues to reconstruct candidate codewords using G . The indistinguishability of this and the previous hybrid follows directly from the PRG security of G and is proven in Lemma 4.5.

Hybrid $\mathcal{H}_4$: This hybrid proceeds as $\mathcal{H}_3$ except that $\mathbf{c}_1^*$ is replaced by an independent uniform codeword in Σ^n . The indistinguishability of this and the previous hybrid follows from the CPA pseudorandomness of PRC_{cpa} and is proven in Lemma 4.6.

Hybrid $\mathcal{H}_5$: This hybrid proceeds as $\mathcal{H}_4$ except that the decoder oracle uses the $b = 1$ path, i.e., $\text{Decode}_0^\emptyset$ is replaced by $\text{Decode}_1^\emptyset$ in the pre-challenge phase and $\text{Decode}_0^{\{\mathbf{c}_{\text{cca}}^*\}}$ by $\text{Decode}_1^{\{\mathbf{c}_{\text{cca}}^*\}}$ in the post-challenge phase. The indistinguishability of this and the previous hybrid follows from the adaptive robustness of PRC_{cpa} and is proven in Lemma 4.7.

Hybrid $\mathcal{H}_6$: This hybrid proceeds as $\mathcal{H}_5$ except that $\mathbf{c}_0^*$ is replaced by an independent uniform codeword in Σ^n . The indistinguishability of this and the previous hybrid follows from the CPA pseudorandomness of PRC_{cpa} and is proven in Lemma 4.8.

Hybrid $\mathcal{H}_7$: This hybrid proceeds as $\mathcal{H}_6$ except that the decoder oracle uses the $b = 0$ path again, i.e., $\text{Decode}_1^\emptyset$ is replaced by $\text{Decode}_0^\emptyset$ in the pre-challenge phase and $\text{Decode}_1^{\{\mathbf{c}_{\text{cca}}^*\}}$ by $\text{Decode}_0^{\{\mathbf{c}_{\text{cca}}^*\}}$ in the post-challenge phase. The challenge-ciphertext filter remains in place. The indistinguishability of this and the previous hybrid follows from the adaptive robustness of PRC_{cpa} and is proven in Lemma 4.7. From this hybrid onward, the oracle uses sk_0 only, so sk_1 is no longer needed.

Hybrid $\mathcal{H}_8$: This hybrid proceeds as $\mathcal{H}_7$ except that the post-challenge oracle uses $\text{Decode}_0^\emptyset$ instead of $\text{Decode}_0^{\{\mathbf{c}_{\text{cca}}^*\}}$, i.e., the challenge-ciphertext filter is removed. The indistinguishability of this and the previous hybrid follows from almost perfect correctness and IND-CPA security of PKE and is proven in Lemma 4.9. In this hybrid the challenge $\mathbf{c}^*$ is uniform in Σ^{2n} , the latent ciphertext $\mathbf{c}_{\text{cca}}^*$ and sk_1 are unused, and the oracle is the real decoder $\text{Decode}_0^\emptyset = \text{Decode}(\text{sk}, \cdot)$ with the outer exclusion test. Hence this hybrid is exactly $\text{Exp}_{\text{PRC}_{\text{cca}}, \mathcal{A}, \beta}^{\text{cca2}, 1}$.

Combining the lemmas below with the triangle inequality, we conclude that for every PPT adversary $\mathcal{A}$ there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$,

$$|\Pr[\mathcal{H}_0(\lambda, \mathcal{A}) = 1] - \Pr[\mathcal{H}_8(\lambda, \mathcal{A}) = 1]| \leq \text{negl}(\lambda),$$

where $\mathcal{H}_i(\lambda, \mathcal{A})$ denotes the output of $\mathcal{A}$ in hybrid $\mathcal{H}_i$. Since $\mathcal{H}_0 = \text{Exp}_{\text{PRC}_{\text{cca}}, \mathcal{A}, \beta}^{\text{cca2}, 0}$ and $\mathcal{H}_8 = \text{Exp}_{\text{PRC}_{\text{cca}}, \mathcal{A}, \beta}^{\text{cca2}, 1}$, this bounds $\text{Adv}_{\text{PRC}_{\text{cca}}, \mathcal{A}, \beta}^{\text{cca2}}(\lambda)$, proving the theorem. $\square$ $\square$

Lemma 4.3 ($\mathcal{H}_0 \approx \mathcal{H}_1$). *Assuming PKE is almost perfectly correct, then, for every PPT adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for all λ , $|\Pr[\mathcal{H}_0(\lambda, \mathcal{A}) = 1] - \Pr[\mathcal{H}_1(\lambda, \mathcal{A}) = 1]| \leq \text{negl}(\lambda)$.*

Proof. By almost perfect correctness, except with negligible probability over **Setup** the pair $(\text{pk}_{\text{cca}}, \text{sk}_{\text{cca}})$ is one for which $\text{PKE.Dec}(\text{sk}_{\text{cca}}, \cdot)$ correctly decrypts all encryptions under pk_{cca} , we denote this event as **Good**. We show that conditioned on **Good** the two hybrids are identical.

The hybrids differ only in the post-challenge oracle, which uses $\text{Decode}_0^\emptyset$ in $\mathcal{H}_0$ and $\text{Decode}_0^{\{\mathbf{c}_{\text{cca}}^*\}}$ in $\mathcal{H}_1$. Fix a post-challenge query $\tilde{\mathbf{c}}$ with $\text{Ham}(\tilde{\mathbf{c}}, \mathbf{c}^*) > \beta N$ (otherwise both oracles return $\perp$ by the outer exclusion test). Let $\mathbf{z} := \text{PRC}_{\text{cpa}}.\text{Decode}(\text{sk}_0, \tilde{\mathbf{c}}_0)$. The two decoders act identically unless $\mathbf{z} = \mathbf{c}_{\text{cca}}^*$, the only value the filter excludes. Suppose $\mathbf{z} = \mathbf{c}_{\text{cca}}^*$. Due to **Good**, decryption gives $\mathbf{x}^* = (\text{msg}^*, \mathbf{s}_0^*, \mathbf{s}_1^*)$, so the reconstruction yields $\hat{\mathbf{c}}_b = \text{PRC}_{\text{cpa}}.\text{Encode}(\text{pk}_b, \mathbf{c}_{\text{cca}}^*; G(\mathbf{s}_b^*)) = \mathbf{c}_b^*$ and hence $\hat{\mathbf{c}} = \mathbf{c}^*$. Then $\text{Decode}_0^\emptyset(\tilde{\mathbf{c}})$ accepts only if $\text{Ham}(\mathbf{c}^*, \tilde{\mathbf{c}}) \leq t \leq \beta N$, contradicting $\text{Ham}(\tilde{\mathbf{c}}, \mathbf{c}^*) > \beta N$. Thus $\text{Decode}_0^\emptyset(\tilde{\mathbf{c}})$ returns $\perp$, exactly as $\text{Decode}_0^{\{\mathbf{c}_{\text{cca}}^*\}}(\tilde{\mathbf{c}})$ does via the filter. Hence the two oracles agree on **Good**, and $|\Pr[\mathcal{H}_0 = 1] - \Pr[\mathcal{H}_1 = 1]| \leq \Pr[\neg \text{Good}] = \text{negl}(\lambda)$. $\square$ $\square$

Lemma 4.4 ($\mathcal{H}_1 \approx \mathcal{H}_2$). *Assuming PKE is IND-CCA2-secure, then, for every PPT adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for all λ , $|\Pr[\mathcal{H}_1(\lambda, \mathcal{A}) = 1] - \Pr[\mathcal{H}_2(\lambda, \mathcal{A}) = 1]| \leq \text{negl}(\lambda)$.*

Proof. To prove this lemma, we build a PPT adversary $\mathcal{B}$ against the IND-CCA2 security of PKE from any $\mathcal{A}$ distinguishing $\mathcal{H}_1$ and $\mathcal{H}_2$. On input pk_{cca} and with access to a decryption oracle $\text{PKE.Dec}(\text{sk}_{\text{cca}}, \cdot)$, $\mathcal{B}$ samples $(\text{pk}_b, \text{sk}_b) \leftarrow \text{PRC}_{\text{cpa}}.\text{Setup}(1^\lambda, \ell_{\text{ct}})$ for $b \in \{0, 1\}$, sets $\text{pk} = (\text{pk}_0, \text{pk}_1, \text{pk}_{\text{cca}})$, and runs $\mathcal{A}$.

$\mathcal{B}$ answers each pre-challenge decoder query $\tilde{\mathbf{c}}$ by simulating $\text{Decode}_0^\emptyset(\tilde{\mathbf{c}})$. It computes $\mathbf{z} \leftarrow \text{PRC}_{\text{cpa}}.\text{Decode}(\text{sk}_0, \tilde{\mathbf{c}}_0)$, returns $\perp$ if $\mathbf{z} = \perp$, and otherwise obtains $\mathbf{x}$ by querying its own decryption oracle on $\mathbf{z}$ (rather than using sk_{cca} , which it does not hold), then finishes the decoder computation and applies the global check.

When $\mathcal{A}_1$ outputs msg^* , $\mathcal{B}$ samples $\mathbf{s}_0^*, \mathbf{s}_1^* \leftarrow \{0, 1\}^\lambda$ and submits the equal-length challenge messages $(\text{msg}^*, \mathbf{s}_0^*, \mathbf{s}_1^*)$ and $\mathbf{x}_0$, receiving a challenge ciphertext $\mathbf{c}_{\text{cca}}^*$. It forms $\mathbf{c}_b^* := \text{PRC}_{\text{cpa}}.\text{Encode}(\text{pk}_b, \mathbf{c}_{\text{cca}}^*; G(\mathbf{s}_b^*))$ and returns $\mathbf{c}^* := (\mathbf{c}_0^*, \mathbf{c}_1^*)$. On each post-challenge query $\tilde{\mathbf{c}}$, $\mathcal{B}$ first applies the outer exclusion test, returning $\perp$ if $\text{Ham}(\tilde{\mathbf{c}}, \mathbf{c}^*) \leq \beta N$. Otherwise it computes $\mathbf{z} \leftarrow \text{PRC}_{\text{cpa}}.\text{Decode}(\text{sk}_0, \tilde{\mathbf{c}}_0)$, returns $\perp$ if $\mathbf{z} = \perp$ or $\mathbf{z} = \mathbf{c}_{\text{cca}}^*$ (realizing the filter $F = \{\mathbf{c}_{\text{cca}}^*\}$), and otherwise queries its decryption oracle on $\mathbf{z} \neq \mathbf{c}_{\text{cca}}^*$ to finish the decoder computation. Since $\mathbf{z} \neq \mathbf{c}_{\text{cca}}^*$ in every post-challenge oracle call, $\mathcal{B}$ never queries its decryption oracle on the challenge ciphertext, so the reduction is valid.

If the challenge ciphertext encrypts $(\text{msg}^*, \mathbf{s}_0^*, \mathbf{s}_1^*)$, then $\mathcal{B}$ perfectly simulates $\mathcal{H}_1$. If it encrypts $\mathbf{x}_0$, then $\mathcal{B}$ perfectly simulates $\mathcal{H}_2$. Finally $\mathcal{B}$ outputs $\mathcal{A}$'s guess. Hence $|\Pr[\mathcal{H}_1 = 1] - \Pr[\mathcal{H}_2 = 1]| \leq \text{Adv}_{\text{PKE}, \mathcal{B}}^{\text{cca2}}(\lambda) = \text{negl}(\lambda)$. $\square$ $\square$

Lemma 4.5 ($\mathcal{H}_2 \approx \mathcal{H}_3$). *Assuming G is a secure PRG, then, for every PPT adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for all λ , $|\Pr[\mathcal{H}_2(\lambda, \mathcal{A}) = 1] - \Pr[\mathcal{H}_3(\lambda, \mathcal{A}) = 1]| \leq \text{negl}(\lambda)$.*

Proof. To prove this lemma, we build a PPT adversary $\mathcal{B}$ against the PRG security of G from any $\mathcal{A}$ distinguishing $\mathcal{H}_2$ and $\mathcal{H}_3$. On input 1^λ and with access to $\mathcal{O}_b^{\text{prg}}$, $\mathcal{B}$ samples both underlying-PRC key pairs and the PKE key pair, gives $\text{pk} = (\text{pk}_0, \text{pk}_1, \text{pk}_{\text{cca}})$ to $\mathcal{A}$, and simulates all decoder queries using Decode_0^F . This simulation requires only sk_0 and sk_{cca} , both of which $\mathcal{B}$ knows. The filter is $F = \emptyset$ before the challenge and $F = \{\mathbf{c}_{\text{cca}}^*\}$ afterwards; in the post-challenge phase, $\mathcal{B}$ first applies the outer exclusion test.

When $\mathcal{A}$ outputs msg^* , $\mathcal{B}$ samples $\mathbf{r}_{\text{pke}}^* \leftarrow \{0, 1\}^{\ell_{\text{rand}}}$ and sets

$$\mathbf{c}_{\text{cca}}^* := \text{PKE.Enc}(\text{pk}_{\text{cca}}, \mathbf{x}_0; \mathbf{r}_{\text{pke}}^*).$$

It then makes two oracle calls $\mathbf{y}_0^* \leftarrow \mathcal{O}_b^{\text{prg}}()$ and $\mathbf{y}_1^* \leftarrow \mathcal{O}_b^{\text{prg}}()$, and sets

$$\mathbf{c}_j^* := \text{PRC}_{\text{cpa}}.\text{Encode}(\text{pk}_j, \mathbf{c}_{\text{cca}}^*; \mathbf{y}_j^*) \quad \text{for } j \in \{0, 1\}.$$

$\mathcal{B}$ gives $\mathbf{c}^* := (\mathbf{c}_0^*, \mathbf{c}_1^*)$ to $\mathcal{A}$, continues the decoder-oracle simulation, and outputs $\mathcal{A}$'s output bit. Notice that the sampling oracle is needed only to generate the two challenge blocks; the simulated decoder continues to evaluate G itself, exactly as in both hybrids.

If $b = 0$, then $\mathbf{y}_0^*$ and $\mathbf{y}_1^*$ are independent values $G(\mathbf{s}_0^*)$ and $G(\mathbf{s}_1^*)$ for fresh uniform seeds, so $\mathcal{B}$ perfectly simulates $\mathcal{H}_2$. If $b = 1$, then $\mathbf{y}_0^*, \mathbf{y}_1^* \leftarrow \{0, 1\}^{\ell_{\text{Enc}}}$ are independent and uniform, so $\mathcal{B}$ perfectly simulates $\mathcal{H}_3$. Consequently, $|\Pr[\mathcal{H}_2 = 1] - \Pr[\mathcal{H}_3 = 1]| = \text{Adv}_{G, \mathcal{B}}^{\text{prg}}(\lambda) = \text{negl}(\lambda)$. $\square$ $\square$

Lemma 4.6 ($\mathcal{H}_3 \approx \mathcal{H}_4$). *Assuming PRC_{cpa} is CPA-pseudorandom, then, for every PPT adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for all λ , $|\Pr[\mathcal{H}_3(\lambda, \mathcal{A}) = 1] - \Pr[\mathcal{H}_4(\lambda, \mathcal{A}) = 1]| \leq \text{negl}(\lambda)$.*

Proof. To prove this lemma, we build a two-stage PPT adversary $\mathcal{B} = (\mathcal{B}_1, \mathcal{B}_2)$ against the CPA pseudorandomness of PRC_{cpa} from any $\mathcal{A}$ distinguishing $\mathcal{H}_3$ and $\mathcal{H}_4$. On input a challenge public key pk^* , $\mathcal{B}_1$ sets $\text{pk}_1 := \text{pk}^*$, samples $(\text{pk}_0, \text{sk}_0) \leftarrow \text{PRC}_{\text{cpa}}.\text{Setup}(1^\lambda, \ell_{\text{ct}})$ and $(\text{pk}_{\text{cca}}, \text{sk}_{\text{cca}}) \leftarrow \text{PKE.Setup}(1^\lambda, \ell_{\text{msg}} + 2\lambda)$, and runs $\mathcal{A}$, simulating its decoder queries using Decode_0^F . When $\mathcal{A}$ outputs msg^* , $\mathcal{B}_1$ samples $\mathbf{r}_{\text{pke}}^* \leftarrow \{0, 1\}^{\ell_{\text{rand}}}$, sets

$$\mathbf{c}_{\text{cca}}^* := \text{PKE.Enc}(\text{pk}_{\text{cca}}, \mathbf{x}_0; \mathbf{r}_{\text{pke}}^*),$$

and submits $\mathbf{c}_{\text{cca}}^*$ as its underlying-PRC challenge message, together with the state of the simulation.

Upon receiving the challenge codeword $\mathbf{c}_1^*$, $\mathcal{B}_2$ samples $\mathbf{r}_0^* \leftarrow \{0, 1\}^{\ell_{\text{Enc}}}$, forms

$$\mathbf{c}_0^* := \text{PRC}_{\text{cpa}}.\text{Encode}(\text{pk}_0, \mathbf{c}_{\text{cca}}^*; \mathbf{r}_0^*),$$

and gives $\mathbf{c}^* := (\mathbf{c}_0^*, \mathbf{c}_1^*)$ to $\mathcal{A}$. It answers post-challenge decoder queries using $\text{Decode}_0^{\{\mathbf{c}_{\text{cca}}^*\}}$, subject to the outer exclusion test, and outputs $\mathcal{A}$'s output bit.

If the challenge codeword is a real encoding, then $\mathcal{B}$ perfectly simulates $\mathcal{H}_3$. If the challenge codeword is uniform in Σ^n , then $\mathcal{B}$ perfectly simulates $\mathcal{H}_4$. Hence $|\Pr[\mathcal{H}_3 = 1] - \Pr[\mathcal{H}_4 = 1]| \leq \text{Adv}_{\text{PRC}_{\text{cpa}}, \mathcal{B}}^{\text{cpa}}(\lambda) = \text{negl}(\lambda)$. $\square$ $\square$

Lemma 4.7 ($\mathcal{H}_4 \approx \mathcal{H}_5$, $\mathcal{H}_6 \approx \mathcal{H}_7$). *Assume PRC_{cpa} is adaptively α -robust and PKE is almost perfectly correct. Consider two experiments that are identical except that all decoder queries are answered by Decode_0^F in one and by Decode_1^F in the other, with the same phase-dependent filters F and the same outer exclusion test. Then for every PPT adversary $\mathcal{A}$ making at most q decoder queries, the two experiments are indistinguishable. Concretely their outputs differ by at most $2q \cdot \text{Adv}_{\text{PRC}_{\text{cpa}}}^{\text{rob}}(\lambda) + \text{negl}(\lambda)$. In particular this bounds $|\Pr[\mathcal{H}_4 = 1] - \Pr[\mathcal{H}_5 = 1]|$ and $|\Pr[\mathcal{H}_6 = 1] - \Pr[\mathcal{H}_7 = 1]|$.*

Proof. To prove this lemma, we build two PPT adversaries $\mathcal{B}_0, \mathcal{B}_1$ against the adaptive robustness of PRC_{cpa} from any $\mathcal{A}$ distinguishing the experiments using Decode_0^F and Decode_1^F . The only differences between $\mathcal{H}_4$ and $\mathcal{H}_5$, and between $\mathcal{H}_6$ and $\mathcal{H}_7$, are their decoding oracles. It therefore suffices to bound the probability that the two decoders produce different outputs.

Denote by $\mathcal{G}^0$ and $\mathcal{G}^1$ the experiments using Decode_0^F and Decode_1^F , respectively, and couple all their randomness. Call a decoder query *differing* if the two decoders return different answers on it. Because everything else is identical, the two experiments' views coincide up to the first differing query. Let $\mathcal{E}$ be the event that a differing query occurs, and let $\mathcal{E}_j$ be the event that the j -th query is the first differing one. Then $|\Pr[\mathcal{G}^0 = 1] - \Pr[\mathcal{G}^1 = 1]| \leq \Pr[\mathcal{E}] \leq \sum_{j \in [q]} \Pr[\mathcal{E}_j]$. As in Lemma 4.3, let **Good** be the event that the PKE key is perfectly correct, $\Pr[\neg \text{Good}] = \text{negl}(\lambda)$, and we bound each $\Pr[\mathcal{E}_j \wedge \text{Good}]$.

Fix j and condition on $\mathcal{E}_j \wedge \text{Good}$, with $\tilde{\mathbf{c}} = (\tilde{\mathbf{c}}_0, \tilde{\mathbf{c}}_1)$ the j -th query. Since the two decoders disagree, at least one of them returns a message. We consider the two cases.

Case A (Decode_0^F accepts): It recovers $\mathbf{z} := \text{PRC}_{\text{cpa}}.\text{Decode}(\text{sk}_0, \tilde{\mathbf{c}}_0) \notin F$, decrypts it to $(\text{msg}, \mathbf{s}_0, \mathbf{s}_1)$, and passes the global check. In particular,

$$\text{Ham}(\tilde{\mathbf{c}}_1, \text{PRC}_{\text{cpa}}.\text{Encode}(\text{pk}_1, \mathbf{z}; G(\mathbf{s}_1))) \leq t \leq \alpha n.$$

If it held that $\text{PRC}_{\text{cpa}}.\text{Decode}(\text{sk}_1, \tilde{\mathbf{c}}_1) = \mathbf{z}$, then $\text{Decode}_1^F(\tilde{\mathbf{c}})$ would recover the same $\mathbf{z} \notin F$, decrypt it identically (deterministic decryption), reconstruct the same codeword, and pass the same global check, so the two decoders would agree, contradicting that the query is differing. Hence $\text{PRC}_{\text{cpa}}.\text{Decode}(\text{sk}_1, \tilde{\mathbf{c}}_1) \neq \mathbf{z}$, so the tuple $(\mathbf{z}, G(\mathbf{s}_1), \tilde{\mathbf{c}}_1)$ wins the adaptive α -robustness game for pk_1 .

We turn this into a reduction $\mathcal{B}_0$. It receives pk^* from the adaptive-robustness challenger, sets $\text{pk}_1 := \text{pk}^*$, samples $(\text{pk}_0, \text{sk}_0)$ and the PKE keys, and guesses an index $j \leftarrow [q]$. It simulates $\mathcal{G}^0$ (which needs only sk_0 and sk_{cca}). If the j -th query is accepted by $\text{Decode}_0^F(\tilde{\mathbf{c}})$, $\mathcal{B}_0$ outputs $(\mathbf{z}, G(\mathbf{s}_1), \tilde{\mathbf{c}}_1)$ and halts. When the guess is correct and Case A occurs at the first differing query, $\mathcal{B}_0$ wins its game. Thus $\sum_j \Pr[\mathcal{E}_j \wedge \text{Good} \wedge \text{Case A}] \leq q \cdot \text{Adv}_{\text{PRC}_{\text{cpa}}}^{\text{rob}}(\lambda)$.

Case B (Decode_1^F accepts): By the symmetric argument, the recovered tuple $(\mathbf{z}', G(\mathbf{s}_0), \tilde{\mathbf{c}}_0)$ wins the adaptive robustness game for pk_0 .

In this case, the reduction $\mathcal{B}_1$ embeds pk^* as pk_0 , samples $(\text{pk}_1, \text{sk}_1)$ and the PKE keys, simulates $\mathcal{G}^1$ (which needs only sk_1 and sk_{cca}), guesses j , and outputs the recovered tuple. Thus $\sum_j \Pr[\mathcal{E}_j \wedge \text{Good} \wedge \text{Case B}] \leq q \cdot \text{Adv}_{\text{PRC}_{\text{cpa}}}^{\text{rob}}(\lambda)$.

Since at least one case occurs at the first differing query, summing the two bounds and adding $\Pr[\neg \text{Good}]$ gives $\Pr[\mathcal{E}] \leq 2q \cdot \text{Adv}_{\text{PRC}_{\text{cpa}}}^{\text{rob}}(\lambda) + \text{negl}(\lambda)$, which concludes the proof of the lemma. $\square$ $\square$

Lemma 4.8 ($\mathcal{H}_5 \approx \mathcal{H}_6$). *Assuming PRC_{cpa} is CPA-pseudorandom, then, for every PPT adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for all λ , $|\Pr[\mathcal{H}_5(\lambda, \mathcal{A}) = 1] - \Pr[\mathcal{H}_6(\lambda, \mathcal{A}) = 1]| \leq \text{negl}(\lambda)$.*

Proof. To prove this lemma, we build a two-stage PPT adversary $\mathcal{B} = (\mathcal{B}_1, \mathcal{B}_2)$ against the CPA pseudorandomness of PRC_{cpa} from any $\mathcal{A}$ distinguishing $\mathcal{H}_5$ and $\mathcal{H}_6$. On input a challenge public key pk^* , $\mathcal{B}_1$ sets $\text{pk}_0 := \text{pk}^*$, samples $(\text{pk}_1, \text{sk}_1) \leftarrow \text{PRC}_{\text{cpa}}.\text{Setup}(1^\lambda, \ell_{\text{ct}})$ and $(\text{pk}_{\text{cca}}, \text{sk}_{\text{cca}}) \leftarrow \text{PKE}.\text{Setup}(1^\lambda, \ell_{\text{msg}} + 2\lambda)$, and runs $\mathcal{A}$, answering all decoder queries using Decode_1^F . When $\mathcal{A}$ outputs msg^* , $\mathcal{B}_1$ samples $\mathbf{r}_{\text{pke}}^* \leftarrow \{0, 1\}^{\ell_{\text{rand}}}$, sets

$$\mathbf{c}_{\text{cca}}^* := \text{PKE}.\text{Enc}(\text{pk}_{\text{cca}}, \mathbf{x}_0; \mathbf{r}_{\text{pke}}^*),$$

and submits $\mathbf{c}_{\text{cca}}^*$ as its underlying-PRC challenge message, together with the state of the simulation.

Upon receiving the challenge codeword $\mathbf{c}_0^*$, $\mathcal{B}$ samples $\mathbf{c}_1^* \leftarrow \Sigma^n$ uniformly, gives $\mathbf{c}^* := (\mathbf{c}_0^*, \mathbf{c}_1^*)$ to $\mathcal{A}$, and continues the simulation using $\text{Decode}_1^{\{\mathbf{c}_{\text{cca}}^*\}}$, subject to the outer exclusion test. Finally, it outputs $\mathcal{A}$'s output bit.

If the challenge codeword is a real encoding, then $\mathcal{B}$ perfectly simulates $\mathcal{H}_5$. If the challenge codeword is uniform in Σ^n , then $\mathcal{B}$ perfectly simulates $\mathcal{H}_6$. Hence $|\Pr[\mathcal{H}_5 = 1] - \Pr[\mathcal{H}_6 = 1]| \leq \text{Adv}_{\text{PRC}_{\text{cpa}}, \mathcal{B}}^{\text{cpa}}(\lambda) = \text{negl}(\lambda)$. $\square$ $\square$

Lemma 4.9 ($\mathcal{H}_7 \approx \mathcal{H}_8$). *Assuming PKE is almost perfectly correct and IND-CPA-secure, then, for every PPT adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for all λ , $|\Pr[\mathcal{H}_7(\lambda, \mathcal{A}) = 1] - \Pr[\mathcal{H}_8(\lambda, \mathcal{A}) = 1]| \leq \text{negl}(\lambda)$.*

Proof. To prove this lemma, we couple the executions of $\mathcal{H}_7$ and $\mathcal{H}_8$ on the same randomness and bound the advantage of any $\mathcal{A}$ distinguishing them by the probability that their post-challenge oracles diverge. Under this coupling, $\mathcal{H}_7$ and $\mathcal{H}_8$ are identical except for the post-challenge decoder oracle, which is $\text{Decode}_0^{\{\mathbf{c}_{\text{cca}}^*\}}$ in $\mathcal{H}_7$ and $\text{Decode}_0^\emptyset$ in $\mathcal{H}_8$. On a query $\tilde{\mathbf{c}} = (\tilde{\mathbf{c}}_0, \tilde{\mathbf{c}}_1)$, both oracles first apply the outer exclusion test of Definition 3.9, returning $\perp$ whenever $\text{Ham}(\tilde{\mathbf{c}}, \mathbf{c}^*) \leq \beta N$; on a query that passes this test, both compute $\mathbf{z} := \text{PRC}_{\text{cpa}}.\text{Decode}(\text{sk}_0, \tilde{\mathbf{c}}_0)$ and decode it identically, the sole difference being that $\mathcal{H}_7$ instead returns $\perp$ when $\mathbf{z} = \mathbf{c}_{\text{cca}}^*$ (this is what the filter $F = \{\mathbf{c}_{\text{cca}}^*\}$ does). Hence the two oracles return the same answer on every query with $\mathbf{z} \neq \mathbf{c}_{\text{cca}}^*$, and the coupled runs produce the same output unless some post-challenge query has $\mathbf{z} = \mathbf{c}_{\text{cca}}^*$. Denoting the later event with Bad , we get:

$$|\Pr[\mathcal{H}_7 = 1] - \Pr[\mathcal{H}_8 = 1]| \leq \Pr[\text{Bad}].$$

It remains to show that $\Pr[\text{Bad}] = \text{negl}(\lambda)$.

The point is that the outer exclusion test does *not* already forbid such a query, because the challenge codeword $\mathbf{c}^*$ and the latent inner ciphertext $\mathbf{c}_{\text{cca}}^*$ are unrelated. From $\mathcal{H}_6$ onward $\mathbf{c}^* = (\mathbf{c}_0^*, \mathbf{c}_1^*)$ is uniform, while $\mathbf{c}_{\text{cca}}^* = \text{PKE}.\text{Enc}(\text{pk}_{\text{cca}}, \mathbf{x}_0)$ is a fresh encryption of zeros that is never revealed to the adversary and is independent of $\mathbf{c}^*$. A query triggers Bad exactly when its block $\tilde{\mathbf{c}}_0$ lies in the decoding radius of the underlying PRC around an encoding of $\mathbf{c}_{\text{cca}}^*$, i.e., $\tilde{\mathbf{c}}_0$ is close to an honest encoding $\text{PRC}_{\text{cpa}}.\text{Encode}(\text{pk}_0, \mathbf{c}_{\text{cca}}^*; \cdot)$. This honest codeword $\tilde{\mathbf{c}}$ is a function of $\mathbf{c}_{\text{cca}}^*$ alone and bears no relation to the uniform $\mathbf{c}^*$; in particular it is generically far from $\mathbf{c}^*$, so a Bad query easily passes the outer test and is filtered in $\mathcal{H}_7$ but decoded in $\mathcal{H}_8$. Triggering Bad thus requires the adversary to submit a codeword that decodes to the hidden ciphertext $\mathbf{c}_{\text{cca}}^*$, which requires it to predict $\mathbf{c}_{\text{cca}}^*$. In the remainder of the proof we bound this event.

Since the two experiments are identical until Bad , we may evaluate $\Pr[\text{Bad}]$ in $\mathcal{H}_8$. There the oracle $\text{Decode}_0^\emptyset$ does not use $\mathbf{c}_{\text{cca}}^*$ and $\mathbf{c}^*$ is uniform, so the adversary's entire view is independent of the fresh randomness $\mathbf{r}_{\text{pke}}^*$ that form $\mathbf{c}_{\text{cca}}^* = \text{PKE}.\text{Enc}(\text{pk}_{\text{cca}}, \mathbf{x}_0; \mathbf{r}_{\text{pke}}^*)$. We fix $(\text{pk}_{\text{cca}}, \text{sk}_{\text{cca}})$ and all randomness other than $\mathbf{r}_{\text{pke}}^*$; this determines the adversary's at most q post-challenge queries and hence the values $\mathbf{z}_i := \text{PRC}_{\text{cpa}}.\text{Decode}(\text{sk}_0, \tilde{\mathbf{c}}_0^{(i)})$, which are fixed strings independent of $\mathbf{r}_{\text{pke}}^*$. For every fixed $\mathbf{z}_i$, it holds that:

$$\begin{aligned} \Pr_{\mathbf{r}_{\text{pke}}^*}[\mathbf{c}_{\text{cca}}^* = \mathbf{z}_i] &= \Pr_{\mathbf{r}_{\text{pke}}^*}[\text{PKE}.\text{Enc}(\text{pk}_{\text{cca}}, \mathbf{x}_0; \mathbf{r}_{\text{pke}}^*) = \mathbf{z}_i] \\ &\leq p_{\text{max}}(\text{pk}_{\text{cca}}, \mathbf{x}_0). \end{aligned}$$

Since **Bad** occurs only if $\mathbf{z}_i = \mathbf{c}_{\text{cca}}^*$ for some $i \in [q]$, a union bound and then averaging over the key gives

$$\Pr[\text{Bad}] \leq q \cdot \mathbb{E}_{(\mathbf{pk}_{\text{cca}}, \mathbf{sk}_{\text{cca}})} [p_{\max}(\mathbf{pk}_{\text{cca}}, \mathbf{x}_0)] = \text{negl}(\lambda),$$

where the last step follows from Lemma 3.5. Combining the two displayed bounds yields $|\Pr[\mathcal{H}_7 = 1] - \Pr[\mathcal{H}_8 = 1]| \leq \text{negl}(\lambda)$. The argument uses only the IND-CPA security of PKE (through Lemma 3.5), which is implied by its IND-CCA2 security. $\square$ $\square$

4.1 Concrete Instantiation

Theorem 4.2 is generic—it compiles any adaptively robust, CPA-pseudorandom PRC PRC_{cpa} , any secure PRG G , and any almost perfectly correct IND-CCA2-secure PKE PKE into an $\frac{\alpha}{2}$ -CCA2-pseudorandom PRC. All three ingredients are available from standard assumptions.

1. The *underlying PRC* PRC_{cpa} which is an adaptively robust CPA-pseudorandom PRC, can be instantiated from:
 - (a) the Learning Parity with Noise (LPN) assumption [AAC⁺25a],
 - (b) the noisy-McEliece assumption [DJK⁺26a].
2. The *PRG* G can be instantiated from any one-way function [HILL99] and, therefore, a variety of concrete assumptions.
3. The *inner PKE* PKE , an almost perfectly correct IND-CCA2-secure scheme is available from a wide range of assumptions. Direct constructions, each with (almost) perfect correctness, are known from:
 - (a) the Decisional Diffie–Hellman assumption, via the Cramer–Shoup cryptosystem [CS98] and, more generally, hash-proof systems [CS02],
 - (b) the Quadratic Residuosity and Decisional Composite Residuosity assumptions, via the universal hash-proof framework [CS02],
 - (c) the hardness of factoring, via the practical scheme of Hofheinz and Kiltz [HK09],
 - (d) the Learning With Errors assumption [Reg05], via lossy and correlated-input trapdoor functions [PW08, MP12],
 - (e) the (low-noise) Learning Parity with Noise assumption, via the schemes of Kiltz, Masny, and Pietrzak [KMP14] and of Yu and Zhang [YZ16];
 - (f) the McEliece assumption, via the CCA2-secure variant of the McEliece cryptosystem of Döttling et al. [DDMN12].

Some assumptions are not known to yield IND-CCA2 security on their own and instead reach it through a generic transformation applied to a weaker primitive:

- (a) any enhanced trapdoor permutation, or any CPA-secure PKE together with a NIZK proof system, via the double-encryption paradigm of Naor and Yung [NY90] and its adaptively secure realization by Dolev, Dwork, and Naor [DDN91];
- (b) the Computational Diffie–Hellman assumption, and generically any CPA-secure PKE, via the hinting-PRG compiler of Koppula and Waters [KW19];
- (c) any identity-based encryption scheme, via the Canetti–Halevi–Katz transform [CHK04].

Acknowledgement

In preparing this manuscript, the authors used OpenAI Codex (GPT-5.6) for editorial assistance and LaTeX preparation. All AI-assisted material was reviewed, revised, and verified by the authors, who determined the research contributions and technical content and take full responsibility for the manuscript.

Nico Döttling and Mahesh Sreekumar Rajasree: Funded by the European Union (ERC, LACONIC, 101041207). Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or the European Research Council. Neither the European Union nor the granting authority can be held responsible for them.

Venkata Koppula: Funded by the HUJI-IITD Joint Global Research Partnership Program (IRD Project No. IR00016G_FS).

References

- [AAC⁺25a] Omar Alrabiah, Prabhanjan Ananth, Miranda Christ, Yevgeniy Dodis, and Sam Gunn. Ideal pseudorandom codes. In Michal Koucký and Nikhil Bansal, editors, *57th Annual ACM Symposium on Theory of Computing*, pages 1638–1647, Prague, Czechia, June 23–27, 2025. ACM Press.
- [AAC⁺25b] Omar Alrabiah, Prabhanjan Ananth, Miranda Christ, Yevgeniy Dodis, and Sam Gunn. Ideal pseudorandom codes. In *Proceedings of the 57th Annual ACM Symposium on Theory of Computing*, STOC ’25, page 1638–1647, New York, NY, USA, 2025. Association for Computing Machinery.
- [BM84] Manuel Blum and Silvio Micali. How to generate cryptographically strong sequences of pseudorandom bits. *SIAM Journal on Computing*, 13(4):850–864, 1984.
- [CG24] Miranda Christ and Sam Gunn. Pseudorandom error-correcting codes. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology – CRYPTO 2024, Part VI*, volume 14925 of *Lecture Notes in Computer Science*, pages 325–347, Santa Barbara, CA, USA, August 18–22, 2024. Springer, Cham, Switzerland.
- [CGG⁺25] Miranda Christ, Noah Golowich, Sam Gunn, Ankur Moitra, and Daniel Wichs. Improved pseudorandom codes from permuted puzzles. Cryptology ePrint Archive, Report 2025/2222, 2025.
- [CHK04] Ran Canetti, Shai Halevi, and Jonathan Katz. Chosen-ciphertext security from identity-based encryption. In Christian Cachin and Jan Camenisch, editors, *Advances in Cryptology – EUROCRYPT 2004*, volume 3027 of *Lecture Notes in Computer Science*, pages 207–222, Interlaken, Switzerland, May 2–6, 2004. Springer Berlin Heidelberg, Germany.
- [CMY26] Yu Chen, Xinyu Mao, and Hongxu Yi. Advancing pseudorandom codes: Beyond parity checks and standard-model cca1 security. Cryptology ePrint Archive, Paper 2026/1228, 2026.

- [CS98] Ronald Cramer and Victor Shoup. A practical public key cryptosystem provably secure against adaptive chosen ciphertext attack. In Hugo Krawczyk, editor, *Advances in Cryptology – CRYPTO’98*, volume 1462 of *Lecture Notes in Computer Science*, pages 13–25, Santa Barbara, CA, USA, August 23–27, 1998. Springer Berlin Heidelberg, Germany.
- [CS02] Ronald Cramer and Victor Shoup. Universal hash proofs and a paradigm for adaptive chosen ciphertext secure public-key encryption. In Lars R. Knudsen, editor, *Advances in Cryptology – EUROCRYPT 2002*, volume 2332 of *Lecture Notes in Computer Science*, pages 45–64, Amsterdam, The Netherlands, April 28 – May 2, 2002. Springer Berlin Heidelberg, Germany.
- [DDMN12] Nico Döttling, Rafael Dowsley, Jörn Müller-Quade, and Anderson C. A. Nascimento. A CCA2 secure variant of the McEliece cryptosystem. *IEEE Transactions on Information Theory*, 58(10):6672–6680, 2012.
- [DDN91] Danny Dolev, Cynthia Dwork, and Moni Naor. Non-malleable cryptography (extended abstract). In *23rd Annual ACM Symposium on Theory of Computing*, pages 542–552, New Orleans, LA, USA, May 6–8, 1991. ACM Press.
- [DFMS22] Jelle Don, Serge Fehr, Christian Majenz, and Christian Schaffner. Online-extractability in the quantum random-oracle model. In Orr Dunkelman and Stefan Dziembowski, editors, *Advances in Cryptology – EUROCRYPT 2022, Part III*, volume 13277 of *Lecture Notes in Computer Science*, pages 677–706, Trondheim, Norway, May 30 – June 3, 2022. Springer, Cham, Switzerland.
- [DFV26] Victor Dyzeryn, Danilo Francati, and Daniele Venturi. Public-key pseudorandom codes from distorted McEliece assumptions. *Cryptology ePrint Archive, Paper 2026/1184*, 2026.
- [DJK⁺26a] Nico Döttling, Antoine Joux, Venkata Koppula, Mahesh Sreekumar Rajasree, and Hendrik Waldner. Chosen ciphertext secure pseudorandom codes in the standard model, 2026. To appear at CRYPTO 2026; Cryptology ePrint Archive, Paper 2026/1227.
- [DJK⁺26b] Nico Döttling, Antoine Joux, Venkata Koppula, Mahesh Sreekumar Rajasree, and Hendrik Waldner. Chosen ciphertext secure pseudorandom codes in the standard model. *Cryptology ePrint Archive*, 2026.
- [DMR25] Nico Döttling, Anne Müller, and Mahesh Sreekumar Rajasree. Separating pseudorandom codes from local oracles. In Benny Applebaum and Huijia (Rachel) Lin, editors, *TCC 2025: 23rd Theory of Cryptography Conference, Part IV*, volume 16271 of *Lecture Notes in Computer Science*, pages 225–257, Aarhus, Denmark, December 1–5, 2025. Springer, Cham, Switzerland.
- [FO99] Eiichiro Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. In Michael J. Wiener, editor, *Advances in Cryptology – CRYPTO’99*, volume 1666 of *Lecture Notes in Computer Science*, pages 537–554, Santa Barbara, CA, USA, August 15–19, 1999. Springer Berlin Heidelberg, Germany.

- [FO13] Eiichiro Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. *Journal of Cryptology*, 26(1):80–101, January 2013.
- [GG24] Surendra Ghentiyala and Venkatesan Guruswami. New constructions of pseudorandom codes. Cryptology ePrint Archive, Report 2024/1425, 2024.
- [GGW25] Sanjam Garg, Sam Gunn, and Mingyuan Wang. Black-box crypto is useless for pseudorandom codes. In Benny Applebaum and Huijia (Rachel) Lin, editors, *TCC 2025: 23rd Theory of Cryptography Conference, Part IV*, volume 16271 of *Lecture Notes in Computer Science*, pages 205–224, Aarhus, Denmark, December 1–5, 2025. Springer, Cham, Switzerland.
- [GM24] Noah Golowich and Ankur Moitra. Edit distance robust watermarks for language models. Cryptology ePrint Archive, Report 2024/898, 2024.
- [HILL99] Johan Håstad, Russell Impagliazzo, Leonid A. Levin, and Michael Luby. A pseudorandom generator from any one-way function. *SIAM Journal on Computing*, 28(4):1364–1396, 1999.
- [HK09] Dennis Hofheinz and Eike Kiltz. The group of signed quadratic residues and applications. In Shai Halevi, editor, *Advances in Cryptology – CRYPTO 2009*, volume 5677 of *Lecture Notes in Computer Science*, pages 637–653, Santa Barbara, CA, USA, August 16–20, 2009. Springer Berlin Heidelberg, Germany.
- [HLMZ26] Shengtang Huang, Xin Li, Songtao Mao, and Zhaienhe Zhou. High-rate public-key pseudorandom codes for edit errors. *arXiv preprint arXiv:2605.19402*, 2026.
- [HLW12] Susan Hohenberger, Allison B. Lewko, and Brent Waters. Detecting dangerous queries: A new approach for chosen ciphertext security. In David Pointcheval and Thomas Johansson, editors, *Advances in Cryptology – EUROCRYPT 2012*, volume 7237 of *Lecture Notes in Computer Science*, pages 663–681, Cambridge, UK, April 15–19, 2012. Springer Berlin Heidelberg, Germany.
- [KMP14] Eike Kiltz, Daniel Masny, and Krzysztof Pietrzak. Simple chosen-ciphertext security from low-noise LPN. In Hugo Krawczyk, editor, *PKC 2014: 17th International Conference on Theory and Practice of Public Key Cryptography*, volume 8383 of *Lecture Notes in Computer Science*, pages 1–18, Buenos Aires, Argentina, March 26–28, 2014. Springer Berlin Heidelberg, Germany.
- [KW19] Venkata Koppula and Brent Waters. Realizing chosen ciphertext security generically in attribute-based encryption and predicate encryption. In Alexandra Boldyreva and Daniele Micciancio, editors, *Advances in Cryptology – CRYPTO 2019, Part II*, volume 11693 of *Lecture Notes in Computer Science*, pages 671–700, Santa Barbara, CA, USA, August 18–22, 2019. Springer, Cham, Switzerland.
- [LSW26] George Lu, Jad Silbak, and Daniel Wichs. Achieving shannon capacity for computationally bounded errors. 2026.

- [MP12] Daniele Micciancio and Chris Peikert. Trapdoors for lattices: Simpler, tighter, faster, smaller. In David Pointcheval and Thomas Johansson, editors, *Advances in Cryptology – EUROCRYPT 2012*, volume 7237 of *Lecture Notes in Computer Science*, pages 700–718, Cambridge, UK, April 15–19, 2012. Springer Berlin Heidelberg, Germany.
- [Ms09] Steven Myers and abhi shelat. Bit encryption is complete. In *50th Annual Symposium on Foundations of Computer Science*, pages 607–616, Atlanta, GA, USA, October 25–27, 2009. IEEE Computer Society Press.
- [NY90] Moni Naor and Moti Yung. Public-key cryptosystems provably secure against chosen ciphertext attacks. In *22nd Annual ACM Symposium on Theory of Computing*, pages 427–437, Baltimore, MD, USA, May 14–16, 1990. ACM Press.
- [PW08] Chris Peikert and Brent Waters. Lossy trapdoor functions and their applications. In Richard E. Ladner and Cynthia Dwork, editors, *40th Annual ACM Symposium on Theory of Computing*, pages 187–196, Victoria, BC, Canada, May 17–20, 2008. ACM Press.
- [Reg05] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. In Harold N. Gabow and Ronald Fagin, editors, *37th Annual ACM Symposium on Theory of Computing*, pages 84–93, Baltimore, MA, USA, May 22–24, 2005. ACM Press.
- [RS92] Charles Rackoff and Daniel R. Simon. Non-interactive zero-knowledge proof of knowledge and chosen ciphertext attack. In Joan Feigenbaum, editor, *Advances in Cryptology – CRYPTO’91*, volume 576 of *Lecture Notes in Computer Science*, pages 433–444, Santa Barbara, CA, USA, August 11–15, 1992. Springer Berlin Heidelberg, Germany.
- [Yao82] Andrew Chi-Chih Yao. Theory and applications of trapdoor functions (extended abstract). In *23rd Annual Symposium on Foundations of Computer Science*, pages 80–91, Chicago, Illinois, November 3–5, 1982. IEEE Computer Society Press.
- [YZ16] Yu Yu and Jiang Zhang. Cryptography with auxiliary input and trapdoor from constant-noise LPN. In Matthew Robshaw and Jonathan Katz, editors, *Advances in Cryptology – CRYPTO 2016, Part I*, volume 9814 of *Lecture Notes in Computer Science*, pages 214–243, Santa Barbara, CA, USA, August 14–18, 2016. Springer Berlin Heidelberg, Germany.

A Proof of Lemma 3.5

Proof. Write $m := |\mathbf{x}|$, so keys are drawn as $(\text{pk}, \text{sk}) \leftarrow \text{PKE.Setup}(1^\lambda, m)$, and fix an efficiently computable second family $\mathbf{x}' = \mathbf{x}'(\lambda) \in \{0, 1\}^m$ with $\mathbf{x}' \neq \mathbf{x}$; such a family exists because the plaintext space contains at least two messages. For a public key pk , let

$$\text{Col}(\text{pk}) := \Pr_{\mathbf{r}, \mathbf{r}' \leftarrow \{0, 1\}^{\ell_{\text{rand}}}} [\text{PKE.Enc}(\text{pk}, \mathbf{x}; \mathbf{r}) = \text{PKE.Enc}(\text{pk}, \mathbf{x}; \mathbf{r}')]]$$

be the probability that two independent encryptions of $\mathbf{x}$ collide, and write $\text{Col}(\lambda) := \mathbb{E}_{(\text{pk}, \text{sk})} [\text{Col}(\text{pk})]$ for its average over the key. The proof has two parts: we first bound $p_{\max}$ by Col , reducing the

claim to showing that $\text{Col}(\lambda)$ is negligible, and then bound $\text{Col}(\lambda)$ by the advantage of an explicit IND-CPA distinguisher.

For the first part, fix pk and write $q_{\mathbf{z}} := \Pr_{\mathbf{r} \leftarrow \{0,1\}^{\ell_{\text{rand}}}} [\text{PKE}.\text{Enc}(\text{pk}, \mathbf{x}; \mathbf{r}) = \mathbf{z}]$ for $\mathbf{z} \in \{0,1\}^{\ell_{\text{ct}}}$. The $(q_{\mathbf{z}})_{\mathbf{z}}$ form a probability distribution, and two independent encryptions collide exactly when they produce the same $\mathbf{z}$, so

$$\text{Col}(\text{pk}) = \sum_{\mathbf{z} \in \{0,1\}^{\ell_{\text{ct}}}} q_{\mathbf{z}}^2 \geq \left(\max_{\mathbf{z}} q_{\mathbf{z}} \right)^2 = p_{\max}(\text{pk}, \mathbf{x})^2,$$

that is, $p_{\max}(\text{pk}, \mathbf{x}) \leq \sqrt{\text{Col}(\text{pk})}$. Averaging over the key and using concavity of the square root (Jensen's inequality),

$$\mathbb{E}_{(\text{pk}, \text{sk})} [p_{\max}(\text{pk}, \mathbf{x})] \leq \mathbb{E}_{(\text{pk}, \text{sk})} [\sqrt{\text{Col}(\text{pk})}] \leq \sqrt{\text{Col}(\lambda)}.$$

Since the square root of a negligible function is negligible, it remains to prove that $\text{Col}(\lambda)$ is negligible.

For the second part, consider the IND-CPA distinguisher $\mathcal{D}$ that, on input pk , submits the challenge messages $\mathbf{x}_0 := \mathbf{x}$ and $\mathbf{x}_1 := \mathbf{x}'$; on receiving the challenge $\mathbf{c}^* = \text{PKE}.\text{Enc}(\text{pk}, \mathbf{x}_b; \mathbf{r}^*)$ for the secret bit $b \in \{0,1\}$ and fresh $\mathbf{r}^* \leftarrow \{0,1\}^{\ell_{\text{rand}}}$, it draws its own $\mathbf{r} \leftarrow \{0,1\}^{\ell_{\text{rand}}}$, computes $\mathbf{c} := \text{PKE}.\text{Enc}(\text{pk}, \mathbf{x}; \mathbf{r})$, and outputs 1 if $\mathbf{c} = \mathbf{c}^*$ and 0 otherwise. Equivalently, $\mathcal{D}$ tests whether the challenge collides with a fresh, independent encryption of $\mathbf{x}$. It is PPT, uses only pk , and its randomness $\mathbf{r}$ is independent of the challenger's $\mathbf{r}^*$.

If $b = 0$, the challenge is itself an encryption of $\mathbf{x}$, so $\mathbf{c}$ and $\mathbf{c}^*$ are two independent encryptions of $\mathbf{x}$ and

$$\Pr[\mathcal{D} = 1 \mid b = 0] = \mathbb{E}_{(\text{pk}, \text{sk})} [\text{Col}(\text{pk})] = \text{Col}(\lambda).$$

If $b = 1$, the challenge encrypts $\mathbf{x}'$ while $\mathbf{c}$ encrypts $\mathbf{x}$, so a collision would force these two distinct plaintexts to share a ciphertext. Almost perfect correctness rules this out on most keys: there are a negligible function $\nu(\cdot)$ and a set Good_{λ} with $\Pr[(\text{pk}, \text{sk}) \in \text{Good}_{\lambda}] \geq 1 - \nu(\lambda)$ such that, on every $(\text{pk}, \text{sk}) \in \text{Good}_{\lambda}$, decryption is correct for all messages and randomness choices. On such a key, since $\text{PKE}.\text{Dec}(\text{sk}, \cdot)$ is a function, every element of $\text{supp } \text{PKE}.\text{Enc}(\text{pk}, \mathbf{x})$ decrypts to $\mathbf{x}$ and every element of $\text{supp } \text{PKE}.\text{Enc}(\text{pk}, \mathbf{x}')$ decrypts to $\mathbf{x}' \neq \mathbf{x}$, so the two supports are disjoint and no collision occurs. Bounding the collision probability by 1 on the remaining keys,

$$\Pr[\mathcal{D} = 1 \mid b = 1] \leq \Pr[(\text{pk}, \text{sk}) \notin \text{Good}_{\lambda}] \leq \nu(\lambda).$$

Subtracting the two estimates, the advantage of $\mathcal{D}$ satisfies

$$\text{Adv}_{\text{PKE}, \mathcal{D}}^{\text{cpa}}(\lambda) = |\Pr[\mathcal{D} = 1 \mid b = 0] - \Pr[\mathcal{D} = 1 \mid b = 1]| \geq \text{Col}(\lambda) - \nu(\lambda).$$

By IND-CPA security the left-hand side is negligible, so $\text{Col}(\lambda) \leq \text{Adv}_{\text{PKE}, \mathcal{D}}^{\text{cpa}}(\lambda) + \nu(\lambda)$ is negligible as well. Combined with the first part, $\mathbb{E}_{(\text{pk}, \text{sk})} [p_{\max}(\text{pk}, \mathbf{x})] \leq \sqrt{\text{Col}(\lambda)} = \text{negl}(\lambda)$, as claimed. $\square \quad \square$